

Del monolito al **microservicio**

Guía de estudio para la **Unidad I: Arquitecturas de desarrollo web** de *Web client and backend development frameworks*, más todo lo que el programa da por sabido antes del primer día de clase. Escrita para leerse completa una vez y consultarse por partes después.

UNIDAD TEMÁTICA	HORAS TEORÍA	HORAS PRÁCTICA	APRENDIZAJE AUTÓNOMO
I de V	15.0	1.5	5.0
COMPETENCIA Diseñar con microservicios			

Índice de la guía

PARTE 0 • CÓMO LEER ESTA GUÍA

- 0.1 El mapa del terreno
- 0.2 La pregunta clave
- 0.3 Cómo usar el documento

PARTE 1 • LO QUE DEBES VENIR ESTUDIADO

- A HTTP a fondo
- B POO y SOLID
- C Patrones de diseño
- D Datos: SQL y NoSQL
- E Lenguaje y ecosistema

F Herramientas

G Autoevaluación

PARTE 2 · LA UNIDAD I PUNTO POR PUNTO

1.1 Principios comunes

1.2 Monolítica

1.3 N–capas

1.4 Cebolla / limpia

1.5 Hexagonal

1.6 Microservicios

PARTE 3 · DE LA TEORÍA A TU PROYECTO

3.1 Decisiones y esqueleto

PARTE 4 · BANCO DE PREGUNTAS

20 Preguntas con respuesta

PARTE 5 · GLOSARIO

A–Z Términos

PARTE 6 · PLAN DE DOS SEMANAS

14 Calendario y fuentes

PARTE 0

Cómo leer esta guía

Qué contiene, en qué orden usarla y cuál es la única idea que amarra las seis secciones de la Unidad I.

0.1 El mapa del terreno

La Unidad I es, con diferencia, la más pesada del programa en horas de teoría: **15 horas con docente frente a 1.5 de práctica**. Compara eso con la Unidad II (13.5 teoría / 9 práctica) o la Unidad V (4.5 / 4.5). El mensaje está en los números: esta unidad casi no se programa, se *entiende*. Y como no se programa, es donde cae la mayor parte de la evaluación escrita.

Ese desequilibrio genera una trampa muy repetida: el alumno típico de séptimo semestre ya sabe escribir código, así que ojea la Unidad I, piensa «esto es puro dibujito de cajas» y se salta al backend. Luego llega el examen y descubre que le

piden distinguir arquitectura de cebolla de arquitectura hexagonal, o justificar por qué un módulo no puede importar a otro. Peor aún: llega al proyecto final con una estructura de carpetas improvisada que ya no puede reorganizar sin romperlo todo.

DEL PROGRAMA OFICIAL

UNIDAD DE COMPETENCIA – «Diseña una aplicación web con base en la arquitectura de microservicios.»

Fíjate en el verbo: **diseña**, no «implementa». La unidad te pide producir un diseño defendible — diagramas, límites entre módulos, justificación de por qué partiste el sistema donde lo partiste. La implementación viene en las unidades II a V.

Las seis secciones y por qué están en ese orden

El programa no lista las arquitecturas al azar. Van de menos a más separación, y cada una nace de un problema que la anterior no resolvió:

PUNTO	ARQUITECTURA	EL PROBLEMA QUE VIENE A RESOLVER
1.1	Principios comunes	El vocabulario y los criterios que se aplican a todas las demás
1.2	Monolítica	El punto de partida: todo junto. Simple, y por eso mismo se degrada
1.3	N–capas	Poner orden dentro del monolito separando responsabilidades técnicas
1.4	Cebolla / limpia	Las n–capas dejan el negocio colgando de la base de datos. Se invierte
1.5	Hexagonal	La misma inversión, expresada como puertos y adaptadores simétricos
1.6	Microservicios	La separación deja de ser lógica y se vuelve física: procesos distintos

Si puedes contar esa historia — qué falla en cada nivel y qué propone el siguiente — ya entendiste la unidad. Todo lo demás son detalles que se cuelgan de ese esqueleto.

0.2 La única pregunta que hay que traer en la cabeza

Hay una sola pregunta que atraviesa las seis secciones, y si la traes presente todo lo demás se acomoda solo:

LA PREGUNTA

¿Quién depende de quién, y hacia dónde apuntan las flechas?

Toda esta unidad es una discusión sobre la **dirección de las dependencias**. Un módulo A depende de B cuando A no compila, no arranca o no se puede probar sin B. Esa flecha, dibujada de A hacia B, es la unidad mínima de análisis arquitectónico.

- En **n-capas**, las flechas apuntan hacia abajo: la interfaz de usuario depende del negocio, y el negocio depende de la base de datos. La consecuencia es que si cambias de base de datos, el negocio se entera.
- En **cebolla y hexagonal**, las flechas apuntan hacia adentro: todo depende del dominio y el dominio no depende de nada. La consecuencia es que puedes cambiar de base de datos sin tocar una sola regla de negocio.
- En **microservicios**, las flechas dejan de ser importaciones de código y se convierten en llamadas de red. La consecuencia es que ahora las dependencias pueden fallar a mitad de camino, y hay que diseñar para eso.

Cuando el profesor dibuje cajas en el pizarrón, no memorices las cajas. Pregunta dónde están las flechas y hacia dónde apuntan. Es literalmente la respuesta a la mitad de las preguntas de examen posibles.

0.3 Cómo usar el documento

La **Parte 1** es el repaso de prerrequisitos: lo que el programa da por adquirido porque supuestamente lo viste en *Tecnologías para el desarrollo de aplicaciones web* y en *Software quality assurance and design patterns*. Léela antes de que empiece el semestre; es lo que responde literalmente a «qué debo venir estudiado».

La **Parte 2** es la Unidad I completa, punto por punto, siguiendo la numeración oficial del programa. Está escrita para leerse en orden la primera vez.

Las **Partes 3 a 6** son de consulta: cómo aterrizar todo esto en tu proyecto del semestre, un banco de preguntas de examen con respuestas modelo, un glosario y un plan de estudio de dos semanas.

A lo largo del texto hay cuatro tipos de recuadro, y cada uno significa algo distinto:

DEL PROGRAMA OFICIAL

Cita textual del PDF del programa. Es lo que el profesor tiene que cubrir por reglamento.

PARA EL EXAMEN

Algo que se pregunta con mucha frecuencia, o una definición que conviene poder soltar de memoria.

TRAMPA COMÚN

Un error de comprensión que casi todos cometen. Vale la pena leerlos aunque saltes el resto.

EN TU PROYECTO

Cómo se traduce el concepto en una decisión concreta del proyecto que vas a entregar.

PARTE 1

Lo que debes venir estudiado

El programa declara esta materia consecuente de *Tecnologías para el desarrollo de aplicaciones web* y *Software quality assurance and design patterns*. Nadie va a repasar esto en clase. Aquí está lo que de verdad se usa, ordenado por cuánto duele no saberlo.

1·A HTTP a fondo

Es el prerequisite número uno y el que más gente cree dominar sin dominarlo. Las unidades II y III son prácticamente un curso de HTTP aplicado, y la Unidad I ya lo asume cuando habla de comunicación entre servicios.

El modelo mental correcto

HTTP es un protocolo **petición–respuesta** y **sin estado**. Sin estado significa que el servidor no recuerda nada de una petición a la siguiente: cada petición debe traer consigo todo lo necesario para ser atendida. Las sesiones, las cookies y los tokens existen precisamente porque HTTP no recuerda — son maneras de simular memoria encima de un protocolo amnésico.

Una petición tiene cuatro partes: **método** (el verbo), **URL** (qué recurso), **cabeceras** (metadatos) y **cuerpo** (los datos, opcional). Una respuesta tiene tres: **código de estado**, **cabeceras** y **cuerpo**.

PETICIÓN HTTP CRUDA

```
POST /api/v1/productos HTTP/1.1
Host: api.miapp.com
Content-Type: application/json
Authorization: Bearer eyJhbGciOiJIUzI1NiIsInR5cCI6IkpXVCJ9...
Accept: application/json

{"nombre":"Teclado mecánico","precio":1299.00,"stock":15}
```

RESPUESTA HTTP CRUDA

```
HTTP/1.1 201 Created
Content-Type: application/json
Location: /api/v1/productos/842

{"id":842,"nombre":"Teclado mecánico","precio":1299.00,"stock":15}
```

Métodos: seguridad e idempotencia

Esta tabla se pregunta en examen casi con seguridad. **Seguro** significa que no modifica el estado del servidor. **Idempotente** significa que ejecutarlo N veces deja el sistema igual que ejecutarlo una vez.

MÉTODO	PROPÓSITO	SEGURO	IDEMPOTENTE	CUERPO
GET	Obtener un recurso	Sí	Sí	No
POST	Crear un recurso subordinado	No	No	Sí
PUT	Reemplazar un recurso completo	No	Sí	Sí
PATCH	Modificar parcialmente	No	Depende	Sí
DELETE	Eliminar un recurso	No	Sí	No
HEAD	Como GET pero sin cuerpo	Sí	Sí	No
OPTIONS	Consultar capacidades (CORS)	Sí	Sí	No

PARA EL EXAMEN

«¿Por qué DELETE es idempotente si la segunda vez devuelve 404?» Porque idempotencia habla del **estado del servidor**, no del código de respuesta. Después de uno o de cinco DELETE al mismo recurso, el recurso está borrado. El estado final es idéntico. La respuesta puede diferir.

Códigos de estado

Apréndete las familias y unos quince códigos concretos. El error más común en prácticas es devolver `200 OK` para absolutamente todo, incluyendo errores, con un `{"error": "..."}` en el cuerpo. Eso rompe REST y te lo van a marcar.

FAMILIA	SIGNIFICADO	LOS QUE DEBES CONOCER
1xx	Informativo	100 Continue, 101 Switching Protocols (WebSockets)
2xx	Éxito	200 OK, 201 Created, 202 Accepted, 204 No Content
3xx	Redirección	301 Moved Permanently, 302 Found, 304 Not Modified
4xx	Error del cliente	400 Bad Request, 401 Unauthorized, 403 Forbidden, 404 Not Found, 405 Method Not Allowed, 409 Conflict, 422 Unprocessable Entity, 429 Too Many Requests
5xx	Error del servidor	500 Internal Server Error, 502 Bad Gateway, 503 Service Unavailable, 504 Gateway Timeout

TRAMPA COMÚN

401 no es 403. `401 Unauthorized` significa «no sé quién eres, identifícate» — falta o es inválida la credencial. `403 Forbidden` significa «sé perfectamente quién eres y no tienes permiso» — la credencial es válida pero no alcanza. El nombre del 401 está mal puesto históricamente: debería llamarse *Unauthenticated*.

Cabeceras que vas a usar todo el semestre

- `Content-Type` — qué formato lleva el cuerpo que envió (`application/json` , `multipart/form-data`).
- `Accept` — qué formato quiero de vuelta. Base de la negociación de contenido.
- `Authorization` — la credencial. `Bearer <token>` o `Basic <base64>` .
- `Location` — dónde quedó el recurso recién creado. Acompaña al 201.
- `Cache-Control` , `ETag` , `If-None-Match` — caché y peticiones condicionales.
- `Access-Control-Allow-Origin` y familia — CORS. Vuelve en el punto 3.3.2 del programa.
- `Set-Cookie` / `Cookie` — sesiones. Vuelve en 2.6.1.

CÓMO ESTUDIARLO

Abre las herramientas de desarrollador del navegador, pestaña **Network**, entra a cualquier sitio y lee peticiones reales durante media hora. Mira método, estado, cabeceras y cuerpo. Media hora ahí enseña más HTTP que tres capítulos de libro.

1·B POO y SOLID

Aquí está la llave de toda la Unidad I. Las arquitecturas de cebolla y hexagonal **son** el principio de inversión de dependencias aplicado a nivel de sistema completo. Si SOLID te suena a acrónimo que memorizaste para un examen, los puntos 1.4 y 1.5 te van a parecer misticismo.

Los cuatro pilares, en la versión que importa

- **Encapsulamiento** — el objeto esconde cómo hace las cosas y expone qué puede hacer. Arquitectónicamente: un módulo publica una interfaz pequeña y esconde todo lo demás.

- **Abstracción** — trabajar con el concepto, no con la implementación. Programar contra `Repositorio` y no contra `RepositorioPostgres`.
- **Herencia** — reutilización por especialización. Útil, pero la regla moderna es *preferir composición sobre herencia*.
- **Polimorfismo** — el mismo mensaje produce comportamientos distintos según el objeto que lo recibe. Es el mecanismo técnico que hace posible cambiar de base de datos sin tocar el negocio.

SOLID, con el enfoque arquitectónico

LETRA	PRINCIPIO	QUÉ SIGNIFICA	DÓNDE REAPARECE EN LA UNIDAD
S	Responsabilidad única	Una clase debe tener una sola razón para cambiar	Cómo se separan capas y cómo se parten microservicios
O	Abierto/cerrado	Abierto a extensión, cerrado a modificación	Añadir un adaptador nuevo sin tocar el núcleo
L	Sustitución de Liskov	Un subtipo debe poder usarse donde se espera el tipo base	Cambiar el repositorio real por uno falso en pruebas
I	Segregación de interfaces	Muchas interfaces pequeñas, no una gigante	Definición de puertos en arquitectura hexagonal
D	Inversión de dependencias	Depender de abstracciones, no de concreciones	Es el motor de 1.4, 1.5 y buena parte de 1.6

Inversión de dependencias, con código

Esto merece que lo veas escrito, porque es exactamente el salto entre el punto 1.3 y el 1.4 del programa.

```
// servicio de negocio
import { PostgresClient } from "../db/postgres.js";

export class ServicioPedidos {
  constructor() {
    this.db = new PostgresClient(); // acoplado a Postgres
  }
  async confirmar(idPedido) {
    const filas = await this.db.query(
      "SELECT * FROM pedidos WHERE id = $1", [idPedido]
    );
    // ... regla de negocio mezclada con SQL
  }
}
```

Problemas: no puedes probar `ServicioPedidos` sin una base de datos Postgres levantada; si migras a MongoDB reescribes la clase; y una regla de negocio («confirmar un pedido») quedó enredada con detalles de almacenamiento.

```
// dominio/RepositorioPedidos.js – la ABSTRACCIÓN vive en el dominio
export class RepositorioPedidos {
  async porId(id) { throw new Error("no implementado"); }
  async guardar(pedido) { throw new Error("no implementado"); }
}

// dominio/ServicioPedidos.js – no importa nada de infraestructura
export class ServicioPedidos {
  constructor(repositorio) { // se lo inyectan
    this.repositorio = repositorio;
  }
  async confirmar(idPedido) {
    const pedido = await this.repositorio.porId(idPedido);
    pedido.confirmar(); // la regla vive en la entidad
    await this.repositorio.guardar(pedido);
    return pedido;
  }
}

// infraestructura/RepositorioPedidosPostgres.js – IMPLEMENTA la abstracción
import { RepositorioPedidos } from "../dominio/RepositorioPedidos.js";

export class RepositorioPedidosPostgres extends RepositorioPedidos {
  constructor(cliente) { super(); this.cliente = cliente; }
  async porId(id) { /* SQL aquí */ }
  async guardar(p) { /* SQL aquí */ }
}
```

Ahora la flecha de dependencia se invirtió: `infraestructura` importa de `dominio`, y `dominio` no importa de nadie. Puedes probar `ServicioPedidos` pasándole un repositorio falso en memoria, y puedes cambiar Postgres por Mongo escribiendo una clase nueva sin tocar una línea del negocio.

PARA EL EXAMEN

Si te piden explicar la arquitectura de cebolla o la hexagonal y no mencionas inversión de dependencias, la respuesta está incompleta por definición. Las dos *son* ese principio escalado a nivel de sistema.

1.C Patrones de diseño que se usan de verdad

De los veintitrés patrones GoF, en esta materia se usan un puñado una y otra vez. Estos son los que conviene traer frescos, con el matiz de para qué sirven *aquí*.

PATRÓN	QUÉ HACE	DÓNDE APARECE EN LA MATERIA
Repository	Abstrae el acceso a datos como si fuera una colección en memoria	Punto 1.3.1 y 1.4.3. El patrón central de la unidad
DAO	Encapsula operaciones sobre una tabla concreta	Capa de acceso a datos. Emparentado con Repository, no idéntico
Adapter	Traduce una interfaz a otra que el cliente espera	Punto 1.5.4. Le da nombre a media arquitectura hexagonal
DTO	Objeto plano para transportar datos entre capas o procesos	Fronteras entre capas y cuerpos JSON de la API
Facade	Una interfaz simple sobre un subsistema complejo	Servicios de aplicación, API Gateway
Strategy	Familia de algoritmos intercambiables en tiempo de ejecución	Políticas de negocio, métodos de pago, esquemas de autenticación
Factory	Crear objetos sin acoplarse a la clase concreta	Construcción de adaptadores según configuración
Observer	Notificar cambios a suscriptores	Comunicación por eventos entre microservicios (1.6.4)
Singleton	Una única instancia global	Pools de conexión. Úsalo poco: complica las pruebas
Proxy	Intermediario que controla el acceso	API Gateway, sidecars, carga perezosa en ORM

Repository vs DAO: la distinción que preguntan

Se confunden constantemente y no son lo mismo. Un **DAO** es una abstracción sobre *la tabla*: `UsuarioDAO` tiene `insertar`, `actualizarPorId`, `borrarPorId`, y habla el lenguaje de la base de datos. Un **Repository** es una abstracción sobre *la colección de objetos de dominio*: `RepositorioPedidos` tiene `porId`, `guardar`, `pendientesDe(cliente)`, y habla el lenguaje del negocio. Un repositorio puede tocar tres tablas para reconstruir un agregado; un DAO normalmente toca una.

Inyección de dependencias

No es un patrón GoF pero es la técnica que hace posible todo lo anterior. Consiste en que un objeto reciba sus colaboradores desde fuera en vez de construirlos. Tres formas, en orden de preferencia:

1. **Por constructor** — la mejor. Las dependencias quedan explícitas y el objeto nunca existe a medias.
2. **Por método setter** — para dependencias opcionales.
3. **Por campo con anotación** — cómoda en Spring (`@Autowired`), pero esconde las dependencias y complica las pruebas.

Spring, NestJS y Angular traen contenedores de inyección integrados. En Express y React se hace a mano, que para efectos de aprender es incluso mejor porque ves el cableado.

1.D Modelo relacional, SQL y NoSQL

La Unidad III es mapeo objeto–relacional, y un ORM no es más que automatización de algo que tú deberías poder escribir a mano. Si no entiendes la relación, no vas a entender el mapeo — vas a estar peleando con el ORM a ciegas.

RELACIONAL: LO MÍNIMO

- Llaves primarias y foráneas, y qué garantiza la integridad referencial.
- `INNER JOIN`, `LEFT JOIN`, y saber cuándo cada uno cambia el resultado.
- Cardinalidades: 1:1, 1:N y N:M, y cómo se representa cada una en tablas. La N:M siempre necesita tabla intermedia — esto reaparece literal en el punto 3.1.4 del programa.
- Normalización hasta 3FN, y por qué a veces se desnormaliza a propósito.
- Índices: para qué sirven y por qué no se ponen en todo.
- Transacciones y ACID. **Atomicidad, Consistencia, Aislamiento, Durabilidad.** Vuelve en 1.6 cuando se discuta por qué los microservicios no pueden usar transacciones distribuidas cómodamente.

NOSQL: LO MÍNIMO

El programa lo pide explícitamente en el punto 4.2.3 y la bibliografía incluye MongoDB. Basta con entender que hay cuatro familias — documental (MongoDB),

clave–valor (Redis), columnar (Cassandra) y de grafos (Neo4j) — que la documental es la que usarás, y que el intercambio de fondo es: ganas flexibilidad de esquema y escalado horizontal, pierdes joins y garantías transaccionales fuertes.

PARA EL EXAMEN

Teorema CAP. En un sistema distribuido puedes garantizar como máximo dos de: Consistencia, Availability (disponibilidad) y Partition tolerance (tolerancia a particiones de red). Como las particiones de red ocurren sí o sí en una red real, P no es negociable: la elección real es entre C y A. Es un clásico de examen en la parte de microservicios.

1·E El lenguaje y su ecosistema

La bibliografía del programa delata dos rutas posibles, y conviene que preguntes cuál usará tu profesor en la primera clase:

RUTA	LIBROS DEL PROGRAMA	STACK TÍPICO
JavaScript	Holmes (MEAN), Zammeti (TypeScript/React/Node/Webpack/Docker), Biswas (React + Firebase)	Node + Express + MongoDB o Postgres + React o Angular
Java	Ottinger (Hibernate 6), Varanasi (Spring REST), Shmeling (Kubernetes Native)	Spring Boot + JPA/Hibernate + Postgres + Angular o React

Si va por JavaScript

- **Asíncrono de verdad:** callbacks, promesas, `async/await`, `Promise.all`, y manejo de errores con `try/catch` dentro de funciones asíncronas. Es la fuente número uno de bugs en prácticas.
- Módulos ES (`import` / `export`) frente a CommonJS (`require`), y por qué mezclarlos causa errores raros.
- Desestructuración, spread/rest, arrow functions, template literals, encadenamiento opcional.
- `fetch` y cómo se manejan respuestas que no son 2xx (pista: `fetch` no lanza excepción en un 404).

- **TypeScript básico:** tipos, interfaces, genéricos simples. Zammeti está en la bibliografía por algo, y en arquitectura hexagonal las interfaces son literalmente los puertos.
- npm: `package.json`, scripts, dependencias vs devDependencies, `package-lock.json`.

Si va por Java

- Interfaces, clases abstractas, genéricos, colecciones.
- Anotaciones y qué significa que un framework las lea en tiempo de ejecución.
- Maven o Gradle: `pom.xml`, dependencias, ciclo de vida de build.
- Fundamentos de Spring: contenedor de beans, `@Component`, `@Service`, `@Repository`, `@RestController`, inyección por constructor.
- JPA básico: `@Entity`, `@Id`, `@OneToMany`, `@ManyToMany`.

RECOMENDACIÓN PRÁCTICA

Si puedes elegir, empieza por **Node + Express**: el cableado queda a la vista y entiendes qué hace cada pieza. Spring Boot hace muchísimo por ti mediante anotaciones, lo cual es genial cuando ya entiendes la arquitectura y confuso cuando no. Aprende el concepto donde se ve, aplícalo después donde está automatizado.

1.F Herramientas que se dan por sabidas

- **Git** — `commit`, `branch`, `merge`, resolver conflictos, `remote`, `push`, `pull`. Vas a entregar por repositorio y probablemente en equipo. Un conflicto de merge sin resolver el día de la entrega es una historia que se repite cada semestre.
- **Línea de comandos** — navegar, variables de entorno, redirección, procesos, puertos ocupados. En Windows conviene tener WSL2 instalado antes de que empiece el semestre; Docker funciona muchísimo mejor ahí.
- **Cliente HTTP** — Postman, Insomnia o la extensión REST Client de VS Code. Es la práctica 4 completa del programa, así que llegar sabiéndolo es un regalo.
- **Herramientas de desarrollador del navegador** — Network, Console, Application (cookies y almacenamiento local). Imprescindibles cuando el frontend no logra hablar con tu API.

- **Variables de entorno y `.env`** — y la regla de oro: credenciales fuera del repositorio. Van a revisar tu Git.

1·G Autoevaluación antes del primer día

Respuestas honestas. Si fallas más de tres, ahí tienes tu plan de preparación antes de que arranque el semestre.

- ☐ ¿Puedo explicar la diferencia entre 401 y 403 sin buscarla?
- ☐ ¿Sé por qué POST no es idempotente y PUT sí?
- ☐ ¿Puedo escribir de memoria una función `async` que llame a una API, maneje el error y devuelva datos tipados?
- ☐ ¿Sé representar una relación N:M en tablas sin pensarlo dos veces?
- ☐ ¿Puedo dar un ejemplo propio de cada letra de SOLID, no el ejemplo del libro?
- ☐ ¿Sé qué es inyección de dependencias y puedo escribirla a mano sin framework?
- ☐ ¿He resuelto un conflicto de merge en Git yo mismo, sin borrar la carpeta y volver a clonar?
- ☐ ¿He hecho una petición POST con cuerpo JSON y cabecera de autorización desde Postman?
- ☐ ¿Sé qué significa que HTTP sea sin estado y cómo se le añade memoria encima?
- ☐ ¿Puedo dibujar la arquitectura del último proyecto que hice, con las flechas de dependencia bien puestas?

PARTE 2

La Unidad I, punto por punto

Sigue la numeración oficial del programa. Cada sección empieza con lo que dice el PDF y continúa con lo que hay que entender detrás de esa línea.

1.1 Principios comunes para las arquitecturas

DEL PROGRAMA OFICIAL

1.1. Principios comunes para las arquitecturas

Una sola línea en el programa, y sin embargo es el cimiento de las otras cinco secciones. Aquí está lo que esa línea contiene.

Qué es arquitectura de software

La definición operativa más útil: la arquitectura es **el conjunto de decisiones de diseño que son caras de cambiar después**. Elegir el nombre de una variable no es arquitectura. Elegir si el sistema será un proceso o veinte sí lo es, porque revertirlo cuesta meses.

La definición formal, si te la piden en examen (IEEE 1471 / ISO 42010): la arquitectura es la organización fundamental de un sistema, encarnada en sus componentes, las relaciones entre ellos y con el entorno, y los principios que guían su diseño y evolución.

Conviene distinguir tres niveles que se confunden:

NIVEL	ALCANCE	EJEMPLO
Arquitectura	El sistema completo y sus límites	«Tres servicios independientes que hablan por HTTP»
Patrón de diseño	Un problema recurrente entre unas cuantas clases	«El acceso a datos se hace vía Repository»
Idioma / técnica	Una línea o un bloque	«Aquí usamos <code>async/await</code> en vez de <code>.then()</code> »

Los principios que se repiten en todas

SEPARACIÓN DE RESPONSABILIDADES

Cada parte del sistema se ocupa de una cosa. La regla práctica: si tienes que explicar qué hace un módulo y usas la palabra «y» más de una vez, probablemente hace demasiado. Es el principio que engendra las capas del punto 1.3.

COHESIÓN ALTA, ACOPLAMIENTO BAJO

Este par es *el* criterio de calidad arquitectónica, y probablemente lo que te van a pedir justificar en el examen.

- **Cohesión** es qué tan relacionadas están entre sí las cosas que viven dentro de un mismo módulo. Alta cohesión = todo lo de dentro trabaja para el mismo propósito.
- **Acoplamiento** es qué tanto un módulo necesita conocer las tripas de otro. Bajo acoplamiento = puedo cambiar el interior de A sin tocar B.

La meta siempre es **cohesión alta y acoplamiento bajo**. Y no son independientes: si partes mal el sistema, bajas la cohesión y subes el acoplamiento a la vez, porque cosas que deberían estar juntas quedan separadas y luego tienen que llamarse constantemente. Esto explica por qué unos microservicios mal divididos son peores que un monolito.

INVERSIÓN DE DEPENDENCIAS

Ya lo viste en la Parte 1. A nivel arquitectónico se enuncia así: **las políticas de alto nivel no deben depender de los detalles de bajo nivel; ambos deben depender de abstracciones**. El «alto nivel» son las reglas de negocio; los «detalles» son la base de datos, el framework web, el proveedor de correo. Es lo que separa 1.3 de 1.4.

OCULTAMIENTO DE INFORMACIÓN

Formulado por David Parnas en 1972 y sigue vigente: un módulo debe esconder las decisiones que tienen más probabilidad de cambiar. La interfaz pública es un contrato; todo lo demás se reserva el derecho de cambiar sin avisar.

DRY, KISS Y YAGNI

- **DRY** (*Don't Repeat Yourself*) — cada pieza de conocimiento debe tener una representación única. Ojo: aplica al *conocimiento*, no al código que se parece. Dos fragmentos idénticos que cambian por razones distintas deben permanecer separados.
- **KISS** (*Keep It Simple*) — la solución más simple que funcione. En esta unidad significa: no montes microservicios para un CRUD de dos entidades.
- **YAGNI** (*You Aren't Gonna Need It*) — no construyas para requisitos hipotéticos.

Atributos de calidad

Son los criterios contra los cuales se juzga una arquitectura. Ninguna arquitectura es buena en abstracto: es buena *para ciertos atributos*. Esto es lo que convierte «¿cuál es mejor?» en una pregunta mal planteada.

ATRIBUTO	PREGUNTA QUE RESPONDE
Mantenibilidad	¿Cuánto cuesta cambiar algo dentro de seis meses?
Escalabilidad	¿Qué pasa si el tráfico se multiplica por cien?
Disponibilidad	¿Qué porcentaje del tiempo está en pie? ¿Qué se cae cuando algo falla?
Rendimiento	¿Cuánto tarda en responder y cuántas peticiones aguanta?
Seguridad	¿Quién puede hacer qué, y cómo se verifica?
Testabilidad	¿Puedo probar el negocio sin levantar media infraestructura?
Portabilidad	¿Puedo moverlo a otro entorno o proveedor?
Observabilidad	Cuando algo falle en producción, ¿podré saber por qué?

ESCALADO VERTICAL FRENTE A HORIZONTAL

Distinción que se pregunta y que reaparece en la Unidad V. **Vertical** es darle más recursos a la misma máquina: más CPU, más RAM. Es simple pero tiene un techo físico y un punto de fallo único. **Horizontal** es poner más máquinas iguales detrás de un balanceador. No tiene techo práctico, pero exige que la aplicación sea *sin estado* — que cualquier instancia pueda atender cualquier petición. Esa exigencia es la que empuja hacia sesiones basadas en token en lugar de sesiones en memoria del servidor, y es una de las razones por las que los microservicios se llevan bien con la nube.

Todo es un intercambio

La respuesta correcta a «¿cuál es la mejor arquitectura?» es siempre **«depende del contexto»**, seguida de los criterios: tamaño y madurez del equipo, complejidad del dominio, requisitos de escalado, presupuesto de operación y plazo. Una respuesta que declare ganador absoluto a los microservicios está mal, y es exactamente el error que el profesor está buscando.

Conway lo formuló en 1967 y sigue siendo incómodamente cierto: **una organización produce diseños que copian su propia estructura de comunicación**. Tres equipos que no se hablan producirán tres módulos con interfaces mal negociadas. La versión moderna — la «maniobra de Conway inversa» — propone organizar los equipos según la arquitectura que quieres obtener. Vale la pena mencionarlo en un examen de arquitectura; casi nadie lo hace.

Vistas arquitectónicas: el modelo 4+1

Propuesto por Philippe Kruchten. Una arquitectura no se describe con un solo diagrama sino con varias vistas, cada una para una audiencia distinta:

- **Lógica** — funcionalidad para el usuario. Diagramas de clases y de paquetes.
- **De procesos** — concurrencia, procesos, hilos, comunicación.
- **De desarrollo** — organización de módulos y carpetas para el programador.
- **Física** — dónde corre cada cosa: servidores, contenedores, red. Diagrama de despliegue.
- **+1: escenarios** — casos de uso que amarran las cuatro vistas anteriores.

Menciona este modelo si te piden «documentar la arquitectura». Y si prefieres algo más moderno y visual, el **modelo C4** (Contexto, Contenedores, Componentes, Código) es el estándar de facto hoy y es más fácil de dibujar.

Deuda técnica

Metáfora de Ward Cunningham: tomar un atajo hoy es pedir prestado tiempo, y se paga con intereses en forma de cambios cada vez más lentos. No toda deuda es mala — a veces salir antes al mercado vale la pena — pero la deuda *no reconocida* siempre lo es. Arquitectónicamente, la deuda más cara es la de límites mal puestos,

porque corregirla implica mover código entre módulos que ya se acostumbraron a conocerse.

1.2 Arquitectura monolítica

DEL PROGRAMA OFICIAL

1.2. Arquitectura monolítica

1.2.1. Contenedores

Definición

Un monolito es una aplicación que se **construye, despliega y ejecuta como una sola unidad**. Un solo artefacto (un `.jar`, un `.war`, una carpeta de Node), un solo proceso, normalmente una sola base de datos.

TRAMPA COMÚN

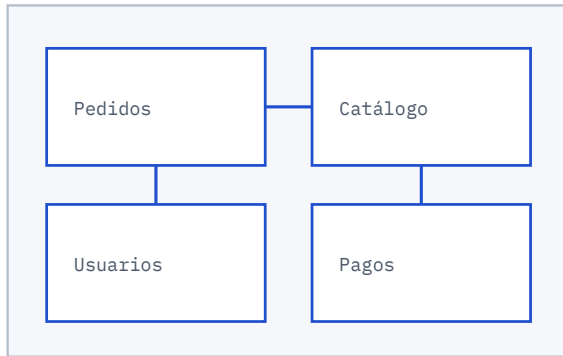
Monolito **no** significa desordenado. Un monolito puede estar impecablemente organizado en capas, con módulos de fronteras estrictas y dominio aislado — toda la Unidad I es aplicable dentro de un monolito. Lo que define al monolito es la **unidad de despliegue**, no la calidad interna. Confundir «monolítico» con «mal hecho» es el error más frecuente de esta sección.

Monolito modular frente a bola de lodo

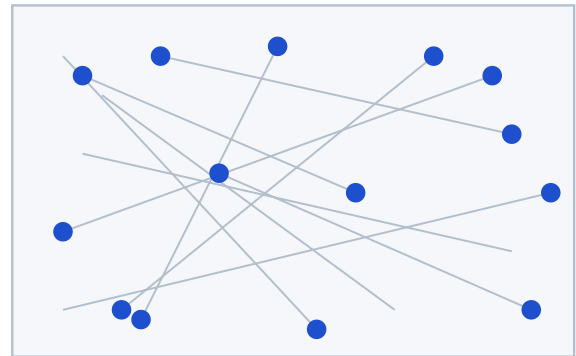
Dos monolitos pueden ser opuestos en calidad:

- El **monolito modular** tiene módulos con límites definidos que se comunican por interfaces explícitas. Un módulo no mete la mano en las tablas de otro. Se despliega junto, pero se razona por separado. Es una arquitectura perfectamente respetable y, para el tamaño de proyecto de esta materia, casi siempre la correcta.
- La **bola de lodo** (*big ball of mud*) no tiene límites: cualquier clase llama a cualquier otra, la lógica de negocio está regada entre controladores, vistas y triggers de base de datos. Es lo que ocurre cuando un monolito crece sin disciplina. No es un estilo arquitectónico, es la ausencia de uno.

Monolito modular



Bola de lodo



Los dos son monolitos: un solo artefacto, un solo despliegue. La diferencia está en si existen límites internos que alguien respeta.

Ventajas y desventajas

A FAVOR	EN CONTRA
Un solo proyecto que clonar, compilar y correr. Un recién llegado es productivo en un día	A cierto tamaño, nadie entiende el sistema completo
Llamadas entre módulos en memoria: rápidas y confiables	Un fallo grave (fuga de memoria, excepción no capturada) tumba todo
Transacciones ACID reales que abarcan todo el sistema	Solo escala verticalmente, o replicando el monolito entero aunque solo un módulo esté saturado
Depuración sencilla: una traza de pila cuenta la historia completa	Todo el sistema queda atado a un único lenguaje y a las versiones de sus librerías
Despliegue simple y barato de operar	Cualquier cambio, por pequeño que sea, obliga a redesplegar todo
Refactorizar entre módulos es fácil: el compilador te avisa	El tiempo de compilación y de pruebas crece hasta volverse molesto

Cuándo elegirlo

Casi siempre al principio. Martin Fowler lo llamó «**monolith first**»: empieza monolítico, descubre dónde están de verdad las fronteras del dominio y extrae servicios cuando exista una razón concreta. La razón es sencilla: los límites entre

microservicios son carísimos de mover una vez trazados, y al inicio de un proyecto todavía no sabes dónde van.

EN TU PROYECTO

Para el proyecto del semestre, salvo que tu profesor exija explícitamente varios servicios, la decisión sólida es **un monolito modular con arquitectura hexagonal por dentro, empaquetado en un contenedor Docker y publicado en un PaaS**. Cumple con la Unidad V, demuestra que entiendes las Unidades I a IV, y no te vas a pasar la mitad del semestre depurando llamadas de red entre tus propios servicios.

1.2.1 Contenedores

Este subpunto confunde a mucha gente porque la palabra «contenedor» significa **dos cosas distintas** en el mundo del backend, y el programa las usa ambas: aquí en 1.2.1 y otra vez en el punto 5.2 («contenedores virtuales de aplicación»).

SENTIDO 1: CONTENEDOR DE APLICACIÓN (SERVIDOR DE APLICACIONES)

El sentido clásico, sobre todo en el mundo Java. Un contenedor de aplicación es un entorno de ejecución que aloja tu aplicación y le proporciona servicios que no tienes que programar: gestión del ciclo de vida, pool de hilos, pool de conexiones, seguridad declarativa, transacciones. Tomcat, Jetty, WildFly, GlassFish. Tú entregas un `.war` y el contenedor lo ejecuta.

La idea clave es la **inversión de control**: tú no controlas el flujo del programa; el contenedor lo controla y llama a tu código cuando corresponde. En Spring Boot esto viene con un Tomcat embebido, por eso puedes correr un `.jar` directamente. En Node no hay contenedor en este sentido: tu proceso *es* el servidor.

SENTIDO 2: CONTENEDOR DE SISTEMA OPERATIVO (DOCKER)

Un empaquetado aislado que contiene tu aplicación con todas sus dependencias y una porción mínima de sistema operativo, ejecutándose sobre el kernel del anfitrión. Es el sentido que domina hoy y el que retomará la Unidad V.

	MÁQUINA VIRTUAL	CONTENEDOR
Qué virtualiza	Hardware completo	El sistema operativo (namespaces y cgroups)
Sistema operativo	Uno completo por VM	Comparte el kernel del anfitrión
Tamaño típico	Gigabytes	Decenas o cientos de megabytes
Arranque	Decenas de segundos	Milisegundos o pocos segundos
Aislamiento	Muy fuerte	Bueno, pero comparte kernel

Lo que resuelve un contenedor es el problema de «en mi máquina sí funciona»: la imagen incluye runtime, librerías y configuración, así que corre igual en tu laptop, en el laboratorio y en el PaaS.

```
DOCKERFILE DE UN MONOLITO NODE
```

```
FROM node:20-alpine
```

```
WORKDIR /app
```

```
# copiar manifiestos primero aprovecha la caché de capas:
```

```
# si package.json no cambió, npm ci no se vuelve a ejecutar
```

```
COPY package*.json ./
```

```
RUN npm ci --omit=dev
```

```
COPY . .
```

```
ENV NODE_ENV=production
```

```
EXPOSE 3000
```

```
# usuario sin privilegios: buena práctica de seguridad
```

```
USER node
```

```
CMD ["node", "src/main.js"]
```

Vocabulario mínimo que hay que manejar: una **imagen** es la plantilla inmutable; un **contenedor** es una instancia en ejecución de esa imagen; un **registro** (Docker Hub, GitHub Container Registry) es donde se publican las imágenes; un **volumen** es almacenamiento que sobrevive al contenedor; y **Docker Compose** orquesta varios contenedores en tu máquina, típicamente aplicación más base de datos.

PARA EL EXAMEN

Si te preguntan por «contenedores» en el contexto de la arquitectura monolítica, la respuesta completa reconoce los dos sentidos: el contenedor de aplicación que aloja y gestiona el ciclo de vida del monolito (Tomcat), y el contenedor de sistema operativo que lo empaqueta para desplegarlo de forma reproducible (Docker). Mencionar ambos, y notar que un monolito en Docker sigue siendo un monolito, es la respuesta que distingue.

1.3 Arquitectura tradicional de n-capas

DEL PROGRAMA OFICIAL

- 1.3. Arquitectura tradicional de n-capas
 - 1.3.1. Capa de acceso a datos
 - 1.3.2. Capa de lógica de negocios
 - 1.3.3. Interfaz de usuario

La forma más difundida de organizar un monolito y, con toda probabilidad, la que ya has usado sin saber cómo se llama. La idea es agrupar el código por **responsabilidad técnica** en capas horizontales, donde cada capa solo puede hablar con la inmediatamente inferior.

Capas y niveles: no son lo mismo

PARA EL EXAMEN

Capa (*layer*) es una división **lógica** del código. **Nivel** (*tier*) es una división **física**: procesos o máquinas distintas. Una aplicación puede tener tres capas y correr en un solo nivel — todo en tu laptop — o tres capas en tres niveles: navegador, servidor de aplicación, servidor de base de datos. En español la distinción se pierde porque ambas se traducen «capa», y es justo por eso que se pregunta.



Las flechas apuntan hacia abajo. El negocio depende del acceso a datos – y ese es exactamente el problema que la arquitectura de cebolla viene a corregir.

1.3.3 Interfaz de usuario (capa de presentación)

Es la frontera con el mundo exterior. Su responsabilidad es traducir entre el protocolo externo y el lenguaje interno de la aplicación; nada más.

- Recibir la petición HTTP y extraer parámetros, cuerpo y cabeceras.
- Validar el *formato* de la entrada – que el correo parezca correo, que la cantidad sea numérica.
- Llamar al servicio de negocio correspondiente.
- Traducir el resultado o la excepción a un código de estado y un cuerpo JSON.

En una API REST esta capa son los *controladores* o *routers*. En una aplicación con vistas del lado del servidor incluye las plantillas. En una SPA, la capa de presentación vive en el navegador y se comunica con el servidor por HTTP – que es exactamente el escenario de la Unidad IV.

TRAMPA COMÚN

Meter reglas de negocio en el controlador. Si en tu `router.post()` aparece un `if` (`usuario.saldo < monto`), esa regla está en el lugar equivocado: pertenece al negocio. La señal de alarma es simple — si cambiaras de HTTP a una interfaz de línea de comandos, ¿qué lógica tendrías que reescribir? Todo eso debía estar más abajo.

Vale la pena distinguir dos tipos de validación, porque se confunden: la **validación de formato** («el precio es un número») es de presentación; la **validación de reglas** («el precio no puede ser menor al costo») es de negocio. La primera protege contra entradas malformadas; la segunda expresa conocimiento del dominio.

1.3.2 Capa de lógica de negocio

El corazón de la aplicación: donde vive el conocimiento que hace que este software valga algo. Reglas, invariantes, cálculos, flujos de trabajo, orquestación de casos de uso, transacciones.

MODELO ANÉMICO FRENTE A MODELO RICO

Una discusión clásica que ilustra el problema de fondo de las n-capas.

En el **modelo anémico**, las entidades son bolsas de datos con getters y setters, y toda la lógica vive en clases de servicio. Es lo que produce naturalmente el estilo n-capas, y funciona bien mientras las reglas son pocas. El problema aparece al crecer: los servicios se vuelven procedimientos gigantes y la misma regla acaba duplicada en tres sitios porque nada impide olvidarla.

En el **modelo rico**, las entidades contienen los datos y las reglas que los gobiernan.

`Pedido` tiene un método `confirmar()` que sabe que un pedido cancelado no puede confirmarse, y no hay manera de saltarse esa regla porque la entidad no lo permite. Es el enfoque de DDD y el que asumen las arquitecturas de los puntos 1.4 y 1.5.

ANÉMICO — LA REGLA SE PUEDE OLVIDAR

```
class Pedido {
  constructor(id, estado, lineas) {
    this.id = id; this.estado = estado; this.lineas = lineas;
  }
}

class ServicioPedidos {
  confirmar(pedido) {
    if (pedido.estado === "CANCELADO") throw new Error("...");
    if (pedido.lineas.length === 0) throw new Error("...");
    pedido.estado = "CONFIRMADO";
  }
}

// nada impide que otro punto del código haga:
// pedido.estado = "CONFIRMADO"; <-- se saltó las reglas
```

RICO — LA REGLA NO SE PUEDE EVITAR

```
class Pedido {
  #estado; // privado de verdad
  constructor(id, lineas) {
    if (lineas.length === 0) throw new PedidoVacio();
    this.id = id; this.lineas = lineas; this.#estado = "BORRADOR";
  }
  get estado() { return this.#estado; }

  confirmar() {
    if (this.#estado === "CANCELADO") throw new PedidoCancelado();
    if (this.#estado === "CONFIRMADO") return; // idempotente
    this.#estado = "CONFIRMADO";
  }
}

// el estado sólo cambia pasando por confirmar()
```

1.3.1 Capa de acceso a datos

Traduce entre objetos y almacenamiento persistente. Su valor está en que el resto del sistema no tenga que saber si detrás hay Postgres, MongoDB o un archivo.

- **Repositorios o DAO** — la interfaz que expone a la capa superior. Ver la distinción en la Parte 1·C.
- **ORM** — Hibernate, TypeORM, Prisma, Sequelize, Mongoose. Es todo el contenido de la Unidad III.

- **Mapeo** — cómo una fila se convierte en objeto y viceversa. Cuidado con el desajuste de impedancia objeto–relacional: la herencia, las colecciones y la identidad no tienen equivalente natural en tablas.
- **Pool de conexiones** — abrir una conexión es caro, así que se reutilizan de un conjunto.
- **Transacciones** — quién las abre y las cierra. Normalmente el servicio de negocio, no el repositorio.

TRAMPA COMÚN

El problema N+1. Cargas 100 pedidos con una consulta, y luego el ORM ejecuta una consulta adicional por cada pedido para traer sus líneas: 101 consultas donde bastaba una con `JOIN`. Es la causa más frecuente de lentitud inexplicable en prácticas con ORM, y en el programa aparece bajo el nombre «estrategias de recuperación de asociaciones» (punto 3.1.5): carga perezosa frente a carga ansiosa.

Reglas de las capas

Una **capa cerrada** obliga a pasar por ella: presentación no puede saltarse el negocio para hablar con datos. Una **capa abierta** se puede sortear. La regla por omisión es cerrada, y el motivo es el aislamiento: si presentación puede tocar datos directamente, la capa de negocio deja de ser el único lugar donde viven las reglas y todo el esquema se degrada.

El precio de las capas cerradas es el llamado *antipatrón del sumidero*: un caso de uso trivial — leer un producto por id — atraviesa tres capas donde dos de ellas no hacen más que reenviar la llamada. Es código repetitivo real, y es un intercambio aceptado a cambio de la consistencia estructural.

Estructura de carpetas típica

ORGANIZACIÓN POR CAPA TÉCNICA

```
src/  
  controllers/          <- presentación  
    productos.controller.js  
    pedidos.controller.js  
  services/             <- negocio  
    productos.service.js  
    pedidos.service.js  
  repositories/         <- acceso a datos  
    productos.repository.js  
    pedidos.repository.js  
  models/  
    producto.model.js  
  middlewares/  
  config/  
  main.js
```

MATIZ QUE SUMA PUNTOS

Esta organización agrupa por *capa técnica*. La alternativa moderna es agrupar por **característica del negocio** — `src/pedidos/{controlador,servicio,repository}.js` — de modo que todo lo relacionado con pedidos vive junto. Esto sube la cohesión, hace evidente dónde están las fronteras del dominio y es el paso natural hacia el monolito modular y, más adelante, hacia los microservicios. Sacar este punto cuando se discuta el 1.3 demuestra que entendiste el principio de cohesión, no solo el diagrama.

La limitación de fondo

Las n-capas ordenan, pero dejan un problema sin resolver: **el negocio depende del acceso a datos**. Consecuencias concretas:

- No puedes probar la lógica de negocio sin una base de datos disponible.
- Un cambio de motor de base de datos se propaga hacia arriba.
- Las entidades del ORM, con sus anotaciones y sus peculiaridades, se filtran hasta el negocio y a veces hasta la respuesta JSON.
- El diseño tiende a empezar por el esquema de tablas en lugar de por el modelo del dominio.

De esa insatisfacción nacen las dos arquitecturas siguientes.

1.4 Arquitectura de cebolla (arquitectura limpia)

DEL PROGRAMA OFICIAL

1.4. Arquitectura de cebolla (arquitectura limpia)

1.4.1. Modelo del dominio

1.4.2. Interfaz de usuario

1.4.3. Infraestructura

1.4.4. Pruebas

De dónde viene

La **arquitectura de cebolla** la propuso Jeffrey Palermo en 2008 como respuesta explícita al problema del punto anterior: en las n-capas, la base de datos termina siendo el centro de gravedad del diseño. Palermo propuso poner en el centro el *modelo del dominio* y dejar todo lo demás como anillos exteriores.

La **arquitectura limpia** (*Clean Architecture*) es la versión que Robert C. Martin popularizó en 2012 y en el libro de 2017. Es esencialmente la misma idea con otros nombres para los anillos. El programa las trata como sinónimos, y para efectos de la materia lo son.

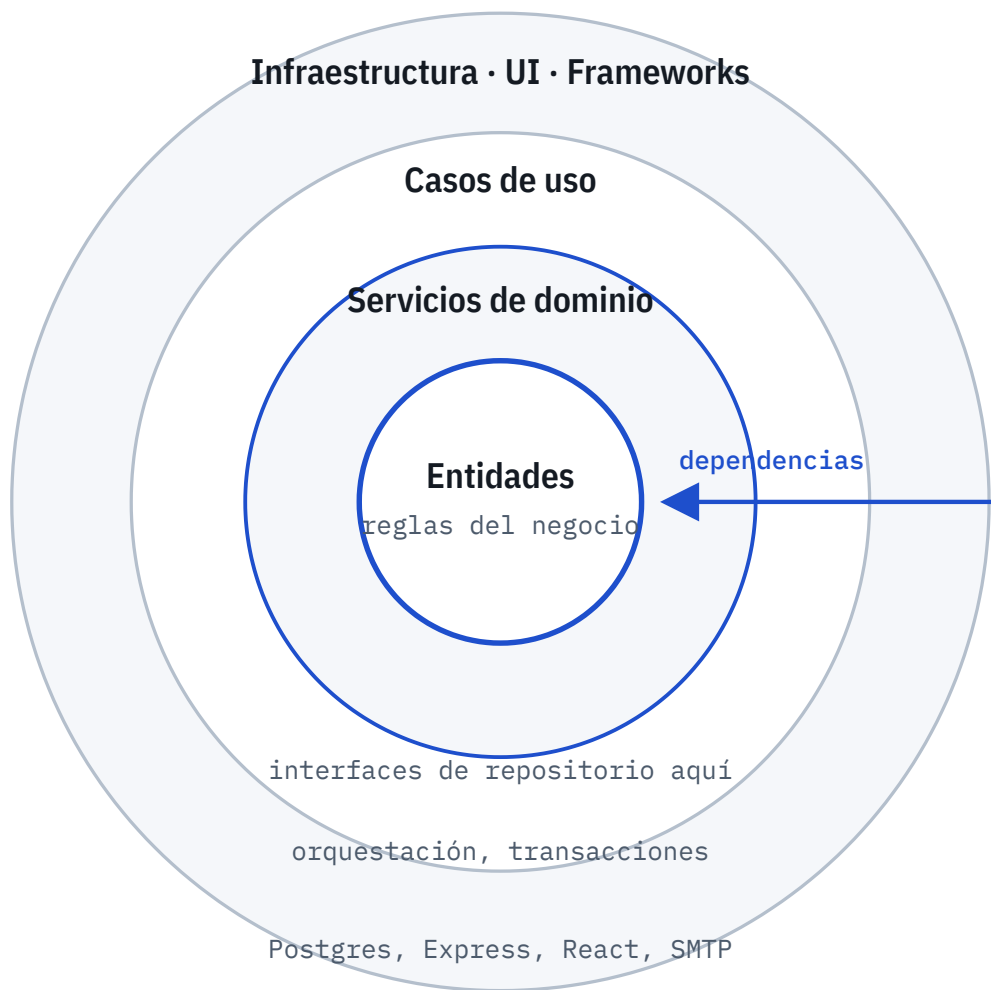
La regla de dependencia

Es la única regla que hay que saberse, y de ella se deduce todo lo demás:

LA REGLA DE DEPENDENCIA

Las dependencias del código fuente solo pueden apuntar *hacia adentro*. Nada en un anillo interior puede saber absolutamente nada de un anillo exterior.

«Nada» es literal: ni un nombre de clase, ni una función, ni una variable, ni un formato de datos. Si el dominio importa `express`, `mongoose` o `@Entity`, la regla está rota.



Todas las flechas apuntan al centro. El anillo exterior conoce al interior; el interior jamás conoce al exterior.

1.4.1 Modelo del dominio

El anillo central. Contiene el conocimiento del negocio que sería cierto aunque el software no existiera.

- **Entidades** — objetos con identidad propia que persiste en el tiempo. Un `Pedido` con id 842 sigue siendo el mismo pedido aunque cambien todos sus datos. La identidad es lo que define a la entidad.
- **Objetos de valor** (*value objects*) — objetos sin identidad, definidos enteramente por sus atributos, e inmutables. `Dinero(1299.00, "MXN")`, `CorreoElectronico("a@b.com")`, un rango de fechas. Dos objetos de valor con los mismos atributos son intercambiables. Usarlos elimina una cantidad

sorprendente de bugs, porque `Dinero` no se puede sumar con un número suelto por accidente.

- **Agregados** — un grupo de entidades y objetos de valor que se trata como una unidad de consistencia, con una entidad raíz que es la única puerta de entrada. `Pedido` es la raíz y sus `LineaDePedido` solo se tocan a través de él. La regla «el total del pedido es la suma de sus líneas» se puede garantizar porque nadie modifica una línea por fuera.
- **Servicios de dominio** — lógica que no cae naturalmente en ninguna entidad. «Transferir de una cuenta a otra» involucra dos cuentas: no es responsabilidad de ninguna de las dos en particular.
- **Interfaces de repositorio** — aquí está el truco central. La *interfaz* del repositorio vive en el dominio porque es el dominio quien declara qué necesita; la *implementación* vive en infraestructura. Esa asimetría es la inversión de dependencias en acción.
- **Excepciones de dominio** — `SaldoInsuficiente`, `PedidoYaCancelado`. Expresan reglas violadas en el lenguaje del negocio, no en el de HTTP.

PARA EL EXAMEN

La prueba de fuego del modelo de dominio: **¿compila y se ejecuta sin base de datos, sin servidor web y sin ningún framework instalado?** Si la respuesta es sí, el dominio está bien aislado. Si necesitas levantar Postgres para instanciar una entidad, no lo está.

Capa de aplicación (casos de uso)

El anillo que rodea al dominio. Aquí viven los **casos de uso**: un caso de uso es una operación completa que el sistema ofrece — «registrar usuario», «confirmar pedido», «generar reporte mensual».

Un caso de uso *orquesta*: pide datos a los repositorios, invoca métodos del dominio, coordina la transacción y devuelve el resultado. Lo que **no** hace es contener reglas de negocio — esas van en las entidades — ni saber nada de HTTP.

APLICACION/CONFIRMARPEDIDO.JS

```
export class ConfirmarPedido {
  constructor(repoPedidos, pasarelaPagos, notificador) {
    this.repoPedidos = repoPedidos;    // interfaz del dominio
    this.pasarelaPagos = pasarelaPagos; // interfaz del dominio
    this.notificador = notificador;    // interfaz del dominio
  }

  async ejecutar({ idPedido, idUsuario }) {
    const pedido = await this.repoPedidos.porId(idPedido);
    if (!pedido) throw new PedidoNoEncontrado(idPedido);
    if (pedido.idUsuario !== idUsuario) throw new AccesoDenegado();

    pedido.confirmar();                // regla en la entidad
    await this.pasarelaPagos.cobrar(pedido.total(), idUsuario);
    await this.repoPedidos.guardar(pedido);
    await this.notificador.pedidoConfirmado(pedido);

    return pedido;
  }
}
```

Observa que en ese archivo no aparece `req`, ni `res`, ni `200`, ni SQL, ni el nombre de ningún proveedor de pagos. Solo conceptos del negocio y abstracciones. Ese es el objetivo.

1.4.2 Interfaz de usuario

Anillo exterior. Aquí sí vive el framework web: rutas, controladores, middlewares, plantillas, o el cliente React de la Unidad IV. Su trabajo es traducir del protocolo al caso de uso y de vuelta.

```

export function montarRutasPedidos(app, casosDeUso) {
  app.post("/api/pedidos/:id/confirmar", async (req, res, next) => {
    try {
      const pedido = await casosDeUso.confirmarPedido.ejecutar({
        idPedido: req.params.id,
        idUsuario: req.usuario.id           // lo puso el middleware de auth
      });
      res.status(200).json(aDTO(pedido)); // traducción a JSON
    } catch (e) {
      next(e);                             // el middleware de errores
    }                                       // traduce excepción -> código HTTP
  });
}

```

Un detalle que suele pasarse por alto: el mapeo de excepciones de dominio a códigos HTTP pertenece a esta capa, no al dominio. `PedidoNoEncontrado` se convierte en 404 *aquí*; el dominio no sabe que existe el 404.

1.4.3 Infraestructura

El otro anillo exterior. Todo lo que es un detalle técnico reemplazable:

- Implementaciones concretas de los repositorios, con su ORM y su SQL.
- Clientes HTTP hacia APIs de terceros — que es la práctica 8 del programa.
- Envío de correo, almacenamiento de archivos, colas de mensajes.
- Configuración, registro de bitácoras, métricas.
- El **ensamblador** (*composition root*): el único punto del programa donde se construyen las clases concretas y se inyectan. Normalmente `main.js` o `Application.java`.

```

import express from "express";
import { Pool } from "pg";
import { RepositorioPedidosPostgres } from "../infraestructura/RepositorioPedidosPostgres";
import { PasarelaStripe } from "../infraestructura/PasarelaStripe.js";
import { NotificadorSMTP } from "../infraestructura/NotificadorSMTP.js";
import { ConfirmarPedido } from "../aplicacion/ConfirmarPedido.js";
import { montarRutasPedidos } from "../infraestructura/http/pedidos.controlador";

const pool = new Pool({ connectionString: process.env.DATABASE_URL });

// adaptadores concretos
const repoPedidos = new RepositorioPedidosPostgres(pool);
const pagos = new PasarelaStripe(process.env.STRIPE_KEY);
const notificador = new NotificadorSMTP(process.env.SMTP_URL);

// casos de uso, cableados con abstracciones
const casosDeUso = {
  confirmarPedido: new ConfirmarPedido(repoPedidos, pagos, notificador)
};

const app = express();
app.use(express.json());
montarRutasPedidos(app, casosDeUso);
app.listen(3000);

```

Para cambiar de Stripe a PayPal escribes `PasarelaPayPal` y modificas una línea de este archivo. Nada más del sistema se entera. Eso es lo que compra la arquitectura.

1.4.4 Pruebas

El programa dedica un subpunto entero a pruebas dentro de esta sección, y no es casualidad: **la testabilidad es el argumento comercial principal de esta arquitectura**. Si te preguntan por qué usar cebolla en vez de n-capas, las pruebas son la mitad de la respuesta.

LA PIRÁMIDE DE PRUEBAS

- **Unitarias** (la base, muchas) — prueban una clase o función aislada. Milisegundos. En esta arquitectura, el dominio y los casos de uso se prueban unitariamente sin infraestructura alguna.
- **De integración** (el medio) — prueban que dos piezas reales encajan: el repositorio contra una base de datos de verdad, por ejemplo.

- **De extremo a extremo** (la punta, pocas) — el sistema completo, por HTTP o por el navegador. Lentas y frágiles, pero son las únicas que prueban que todo junto funciona.

DOBLES DE PRUEBA

Terminología de Gerard Meszaros que se pregunta:

DOBLE	QUÉ HACE
Dummy	Se pasa para rellenar un parámetro, nunca se usa
Stub	Devuelve respuestas fijas preprogramadas
Spy	Es un stub que además registra cómo fue llamado
Mock	Tiene expectativas: falla si no se le llama como se esperaba
Fake	Implementación real pero simplificada: un repositorio en memoria

PRUEBA DE UN CASO DE USO, SIN BASE DE DATOS

```
class RepositorioPedidosEnMemoria { // un FAKE
  constructor(inicial = []) { this.datos = new Map(inicial.map(p => [p.id, p])); }
  async porId(id) { return this.datos.get(id) ?? null; }
  async guardar(p) { this.datos.set(p.id, p); }
}

test("confirmar un pedido lo deja CONFIRMADO y cobra el total", async () => {
  const pedido = new Pedido("842", "u1", [new Linea("sku-1", 2, 100)]);
  const repo = new RepositorioPedidosEnMemoria([pedido]);
  const pagos = { cobrado: null, async cobrar(m) { this.cobrado = m; } }; // SPY
  const notif = { async pedidoConfirmado() {} }; // DUMMY

  const caso = new ConfirmarPedido(repo, pagos, notif);
  const res = await caso.ejecutar({ idPedido: "842", idUsuario: "u1" });

  expect(res.estado).toBe("CONFIRMADO");
  expect(pagos.cobrado).toBe(200);
});
```

Esa prueba corre en milisegundos, no necesita Docker, no necesita red y no falla intermitentemente. En una arquitectura de n-capas clásica, la misma prueba requeriría una base de datos levantada y sembrada.

El costo

TRAMPA COMÚN

Presentar la arquitectura limpia como pura ganancia. Tiene costos reales: más archivos, más interfaces, más indirección y más código de traducción entre entidades de dominio, entidades de ORM y DTOs. Para un CRUD de dos tablas es sobreingeniería evidente. El valor aparece cuando hay reglas de negocio de verdad y el proyecto va a vivir años. Reconocer el intercambio en el examen vale más que recitar las ventajas.

1.5 Arquitectura hexagonal

DEL PROGRAMA OFICIAL

- 1.5. Arquitectura hexagonal
 - 1.5.1. Dominio de la aplicación
 - 1.5.2. Puertos
 - 1.5.3. Capa de servicios
 - 1.5.4. Adaptadores

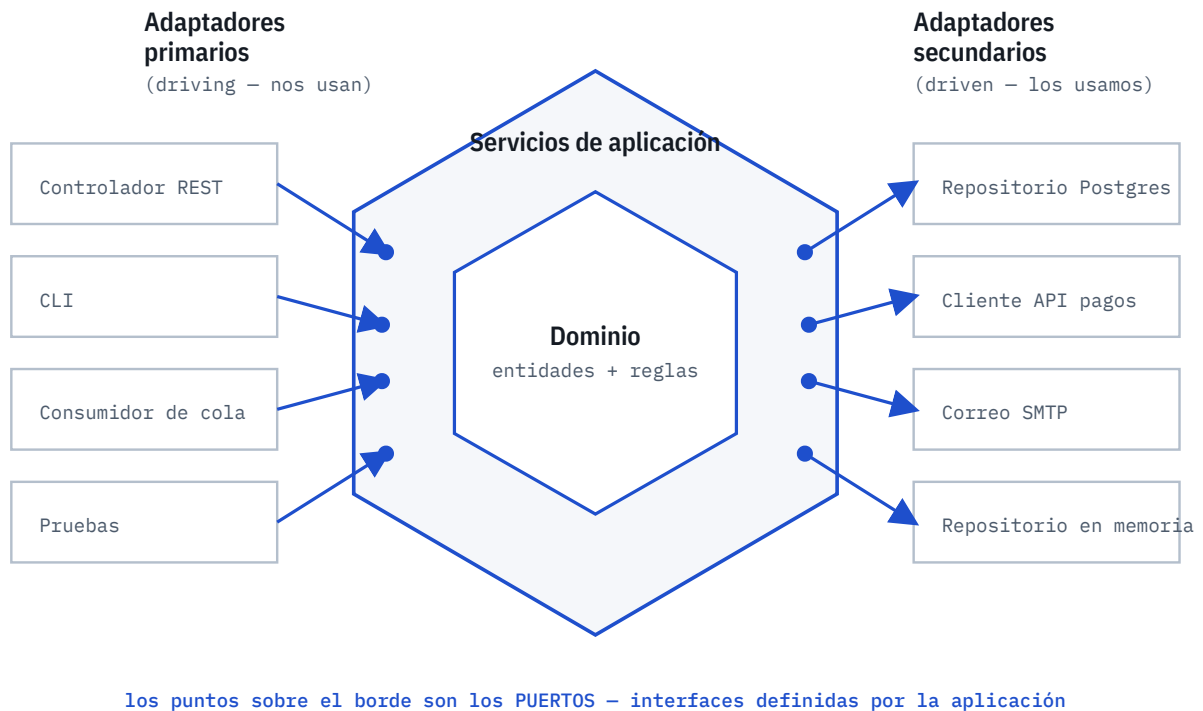
La idea

Alistair Cockburn la formuló en 2005 con el nombre de **Puertos y Adaptadores**, y su enunciado original es una de las frases más claras que se han escrito sobre arquitectura: la aplicación debe poder ser operada indistintamente por usuarios, programas, pruebas automatizadas o scripts, y poder desarrollarse y probarse aislada de sus dispositivos y bases de datos definitivos.

Dicho en corto: **la aplicación no debe saber quién la llama ni a quién llama.**

TRAMPA COMÚN

«¿Por qué un hexágono? ¿Son seis capas?» No. Cockburn eligió el hexágono por una razón puramente de dibujo: quería mostrar que hay *varios* puntos de entrada y salida alrededor del núcleo, y un círculo no deja colocarlos bien. El seis no significa nada. Si alguien te dice que son seis capas, está equivocado, y esta es una pregunta trampa clásica.



Izquierda: alguien nos llama. Derecha: nosotros llamamos a alguien. El hexágono es indiferente a quién hay del otro lado de cada puerto.

1.5.1 Dominio de la aplicación

Es el mismo núcleo que en la arquitectura de cebolla: entidades, objetos de valor, agregados, reglas. Cockburn lo llama simplemente «la aplicación». No conoce ningún framework, ningún protocolo y ninguna base de datos.

1.5.2 Puertos

Un **puerto** es una interfaz que define *la aplicación* para hablar con el exterior. Es un contrato expresado en el lenguaje del dominio, no en el de la tecnología. Hay dos familias, y distinguirlas es lo que se pregunta:

	PUERTOS PRIMARIOS	PUERTOS SECUNDARIOS
Otros nombres	De entrada, <i>driving</i> , conductores	De salida, <i>driven</i> , conducidos
Quién inicia	El exterior llama a la aplicación	La aplicación llama al exterior
Quién define la interfaz	La aplicación	La aplicación
Quién la implementa	La aplicación	Un adaptador de infraestructura
Ejemplo	ConfirmarPedidoPort	RepositorioPedidos, PasarelaPagos

PARA EL EXAMEN

La asimetría es el punto fino: **la aplicación define ambos tipos de puerto, pero solo implementa los primarios**. Los secundarios los implementa infraestructura. Por eso los secundarios son inversión de dependencias pura: el que necesita el servicio dicta la forma del contrato, no el que lo provee.

PUERTOS COMO INTERFACES TYPESCRIPT

```
// ---- puerto PRIMARIO: lo que la aplicación ofrece al mundo ----
export interface ConfirmarPedidoPort {
  ejecutar(cmd: { idPedido: string; idUsuario: string }): Promise<Pedido>;
}

// ---- puertos SECUNDARIOS: lo que la aplicación necesita del mundo ----
export interface RepositorioPedidos {
  porId(id: string): Promise<Pedido | null>;
  guardar(pedido: Pedido): Promise<void>;
}

export interface PasarelaPagos {
  cobrar(monto: Dinero, idUsuario: string): Promise<Comprobante>;
}

export interface Notificador {
  pedidoConfirmado(pedido: Pedido): Promise<void>;
}
```

Nota cómo estas interfaces están escritas en vocabulario de negocio: `cobrar`, `Dinero`, `Comprobante`. Ninguna menciona SQL, HTTP ni Stripe. Un puerto que se

llamara `ejecutarConsultaSQL` no sería un puerto: sería una filtración de infraestructura hacia el dominio.

1.5.3 Capa de servicios

Los **servicios de aplicación** implementan los puertos primarios. Son los mismos casos de uso de la arquitectura de cebolla: orquestan el dominio y los puertos secundarios, gestionan la transacción y no contienen reglas de negocio propias.

SERVICIO DE APLICACIÓN IMPLEMENTANDO EL PUERTO PRIMARIO

```
export class ServicioConfirmarPedido implements ConfirmarPedidoPort {
  constructor(
    private readonly repo: RepositorioPedidos, // puerto secundario
    private readonly pagos: PasarelaPagos, // puerto secundario
    private readonly avisos: Notificador // puerto secundario
  ) {}

  async ejecutar(cmd: { idPedido: string; idUsuario: string }): Promise<Pedido> {
    const pedido = await this.repo.porId(cmd.idPedido);
    if (!pedido) throw new PedidoNoEncontrado(cmd.idPedido);

    pedido.confirmar(); // la REGLA vive en la entidad
    await this.pagos.cobrar(pedido.total(), cmd.idUsuario);
    await this.repo.guardar(pedido);
    await this.avisos.pedidoConfirmado(pedido);
    return pedido;
  }
}
```

1.5.4 Adaptadores

Un **adaptador** es código que conecta un puerto con una tecnología concreta. Es el patrón Adapter del GoF aplicado a la frontera del sistema, y de ahí viene el nombre.

- **Adaptadores primarios** — traducen un estímulo externo en una llamada a un puerto primario. Un controlador REST convierte una petición HTTP en un comando. Un consumidor de cola convierte un mensaje en un comando. Una prueba automatizada también es un adaptador primario — este punto suele sorprender y es exactamente lo que Cockburn quería lograr.
- **Adaptadores secundarios** — implementan un puerto secundario contra una tecnología. `RepositorioPedidosPostgres` implementa `RepositorioPedidos`

usando SQL. `PasarelaStripe` implementa `PasarelaPagos` llamando a la API de Stripe.

DOS ADAPTADORES DEL MISMO PUERTO SECUNDARIO

```
// producción
export class RepositorioPedidosPostgres implements RepositorioPedidos {
  constructor(private readonly pool: Pool) {}

  async porId(id: string): Promise<Pedido | null> {
    const { rows } = await this.pool.query(
      `SELECT p.id, p.usuario_id, p.estado,
        l.sku, l.cantidad, l.precio_unitario
        FROM pedidos p
        LEFT JOIN lineas_pedido l ON l.pedido_id = p.id
        WHERE p.id = $1`, [id]);
    return rows.length ? this.aDominio(rows) : null;    // mapeo fila -> entidad
  }

  async guardar(pedido: Pedido): Promise<void> { /* ... */ }
  private aDominio(rows: any[]): Pedido { /* ... */ }
}

// pruebas: mismo contrato, cero infraestructura
export class RepositorioPedidosEnMemoria implements RepositorioPedidos {
  private datos = new Map<string, Pedido>();
  async porId(id: string)      { return this.datos.get(id) ?? null; }
  async guardar(p: Pedido)     { this.datos.set(p.id, p); }
}
```

Estructura de carpetas

ORGANIZACIÓN HEXAGONAL

```
src/  
  dominio/                                <- NO importa nada de fuera  
    Pedido.ts  
    LineaPedido.ts  
    Dinero.ts  
    errores.ts  
  aplicacion/  
    puertos/  
      entrada/ConfirmarPedidoPort.ts  
      salida/RepositorioPedidos.ts  
      salida/PasarelaPagos.ts  
      salida/Notificador.ts  
    servicios/  
      ServicioConfirmarPedido.ts  
  infraestructura/  
    entrada/  
      http/pedidos.controlador.ts  
      cli/comandos.ts  
    salida/  
      persistencia/RepositorioPedidosPostgres.ts  
      pagos/PasarelaStripe.ts  
      correo/NotificadorSMTP.ts  
  main.ts                                <- el ensamblador
```

EN TU PROYECTO

Si adoptas esta estructura desde el primer commit, ganas dos cosas concretas. Primero, cuando llegue la Unidad III y toque cambiar de acceso directo a ORM, solo tocas `infraestructura/salida/persistencia/`. Segundo, tienes un diagrama que defender en la exposición — y las exposiciones cuentan en la evaluación según el programa. La estructura de carpetas *es* tu argumento arquitectónico hecho visible.

Cebolla, limpia y hexagonal: la comparación

Esta tabla responde la pregunta de examen más previsible de toda la unidad.

	HEXAGONAL (2005)	CEBOLLA (2008)	LIMPIA (2012)
Autor	Alistair Cockburn	Jeffrey Palermo	Robert C. Martin
Metáfora	Hexágono con puertos	Anillos concéntricos	Anillos concéntricos
Énfasis	Simetría entrada/salida y sustituibilidad	El dominio como centro, no la base de datos	La regla de dependencia como ley explícita
Vocabulario	Puertos, adaptadores	Dominio, servicios, infraestructura	Entidades, casos de uso, adaptadores de interfaz, frameworks
Aporte propio	Ver las pruebas como otro adaptador más	Poner las interfaces de repositorio dentro del dominio	Formular la regla en una sola frase

RESPUESTA MODELO

«Las tres pertenecen a la misma familia y comparten el principio central: **el dominio en el centro y las dependencias apuntando hacia adentro**, logrado mediante inversión de dependencias. Difieren en la metáfora y en el énfasis: la hexagonal destaca la simetría entre lo que nos llama y lo que llamamos mediante puertos y adaptadores; la de cebolla destaca que el modelo del dominio, y no la base de datos, es el centro de gravedad; y la limpia enuncia la regla de dependencia de forma explícita y nombra cuatro anillos. En la práctica los equipos las mezclan.»

Señales de que lo estás haciendo mal

- Una entidad de dominio con anotaciones del ORM (`@Entity` , `@Column`) o de serialización. Esas anotaciones son infraestructura.
- Un puerto que devuelve un tipo del ORM, o un `ResultSet` , o un `Response` .
- Un servicio de aplicación que recibe `req` y `res` como parámetros.
- Un `import` desde `dominio/` hacia `infraestructura/` . Con una sola línea así la arquitectura ya no se sostiene.
- Puertos con veinte métodos. Eso viola segregación de interfaces y suele indicar que el agregado está mal delimitado.

TRUCO DE VERIFICACIÓN

Se puede automatizar. En Java existe ArchUnit, que permite escribir una prueba que falla si algún paquete de dominio importa infraestructura. En JavaScript se logra con `eslint-plugin-boundaries` o con reglas de `no-restricted-imports`. Mencionar que las reglas arquitectónicas se pueden *probar automáticamente* es el tipo de detalle que separa una exposición buena de una excelente.

1.6 Arquitectura de microservicios

DEL PROGRAMA OFICIAL

- 1.6. Arquitectura de microservicios
 - 1.6.1. Fundamentos de microservicios
 - 1.6.2. Arquitectura del dominio
 - 1.6.3. Modelo de referencia
 - 1.6.4. Modelo de implementación
 - 1.6.5. Modelo de despliegue
 - 1.6.6. Diseño de aplicaciones web

Es el punto más extenso de la unidad y el que nombra la unidad de competencia. Todo lo anterior fue preparación para esto.

1.6.1 Fundamentos

DEFINICIÓN

Un estilo arquitectónico en el que la aplicación se construye como un conjunto de **servicios pequeños, autónomos y desplegables de forma independiente**, cada uno ejecutándose en su propio proceso y comunicándose por mecanismos ligeros, normalmente HTTP o mensajería asíncrona.

El salto conceptual respecto a todo lo anterior: hasta ahora la separación era **lógica** — carpetas, módulos, interfaces, todo dentro del mismo proceso. Aquí la separación se vuelve **física**. Y esa es la diferencia que lo cambia todo, porque una llamada entre módulos que antes era una llamada a función ahora es una llamada de red que puede tardar, fallar o llegar dos veces.

LAS CARACTERÍSTICAS

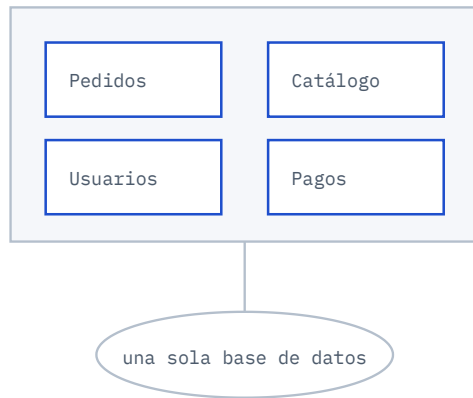
Lewis y Fowler enumeraron nueve rasgos que se citan constantemente. Estos son los que hay que poder nombrar:

1. **Componentización vía servicios** — los componentes son procesos independientes, no librerías enlazadas.
2. **Organización en torno a capacidades de negocio** — el servicio de «pedidos», no el servicio de «base de datos». Un equipo por capacidad, con frontend, backend y datos.
3. **Productos, no proyectos** — el equipo que lo construye lo opera. *You build it, you run it.*
4. **Extremos inteligentes, tuberías tontas** — la lógica vive en los servicios; el medio de transporte solo transporta. Es una reacción explícita contra los buses de servicios empresariales cargados de lógica.
5. **Gobierno descentralizado** — cada servicio puede usar el lenguaje y las herramientas que le convengan. En la práctica se limita, porque cada tecnología extra es un costo permanente.
6. **Gestión de datos descentralizada — una base de datos por servicio.** Esta es la más importante y la que más se incumple.
7. **Automatización de infraestructura** — sin integración y despliegue continuos, esto no es sostenible.
8. **Diseño para el fallo** — los servicios fallan. Hay que asumirlo en el diseño, no tratarlo como excepción.
9. **Diseño evolutivo** — los servicios se reemplazan y se retiran; no son estructuras permanentes.

PARA EL EXAMEN

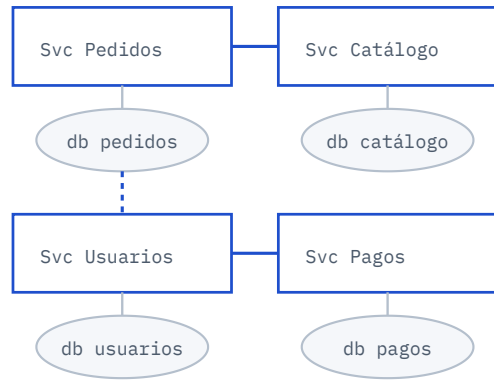
Una base de datos por servicio es el criterio que separa microservicios de verdad de un monolito troceado. Si dos servicios comparten tablas, no son independientes: no puedes cambiar el esquema sin coordinar despliegues, y acabas de construir un *monolito distribuido* — todos los costos de lo distribuido, ninguna de las ventajas. Es el antipatrón que más se pregunta.

Monolito



1 proceso · 1 despliegue
transacciones ACID reales
falla todo o nada

Microservicios



N procesos · N despliegues
consistencia eventual entre servicios
falla parcial: se degrada, no cae

La línea que importa no es la que separa las cajas de arriba, sino la que separa las bases de datos de abajo.

MONOLITO FRENTE A MICROSERVICIOS, SIN PROPAGANDA

CRITERIO	MONOLITO	MICROSERVICIOS
Despliegue	Todo junto, siempre	Cada servicio a su ritmo
Escalado	Se replica el sistema entero	Se escala solo el servicio saturado
Fallo	Se cae todo	Fallo parcial y degradación, si se diseñó para ello
Transacciones	ACID directo	Consistencia eventual, sagas, compensaciones
Tecnología	Una sola pila	Libre por servicio, con el costo que eso implica
Depuración	Una traza de pila	Trazas distribuidas y correlación de bitácoras
Pruebas	Directas	Complejas: contratos, entornos, dobles de servicio
Operación	Barata	Cara: orquestador, registro, monitoreo, CI/CD por servicio
Equipo mínimo	Uno pequeño basta	Varios equipos, o el costo de coordinación te come
Refactorizar límites	Barato: lo hace el IDE	Carísimo: implica migrar datos y coordinar APIs

LAS FALACIAS DE LA COMPUTACIÓN DISTRIBUIDA

Formuladas en Sun Microsystems por Peter Deutsch y otros. Son ocho suposiciones falsas que todo el mundo hace al principio, y explican de dónde sale la dificultad real de los microservicios:

1. La red es confiable.
2. La latencia es cero.
3. El ancho de banda es infinito.
4. La red es segura.
5. La topología no cambia.
6. Hay un solo administrador.
7. El costo de transporte es cero.
8. La red es homogénea.

Cada una de estas falacias tiene un patrón de mitigación correspondiente: reintentos e idempotencia para la primera, caché y agregación para la segunda, TLS y autenticación servicio a servicio para la cuarta, descubrimiento de servicios para la quinta.

1.6.2 Arquitectura del dominio

La pregunta práctica más difícil de todo el tema: **¿dónde se corta?** Y la respuesta que da la industria viene del diseño guiado por el dominio (DDD) de Eric Evans.

CONTEXTO DELIMITADO (BOUNDED CONTEXT)

Es el concepto clave. Un contexto delimitado es una frontera dentro de la cual un modelo y su vocabulario son consistentes y sin ambigüedad. Fuera de esa frontera, las mismas palabras significan otra cosa.

El ejemplo clásico es la palabra *producto*:

- En el contexto de **catálogo**, un producto tiene descripción, imágenes, categoría y atributos de búsqueda.
- En el de **inventario**, un producto es un SKU con existencias, ubicación en almacén y punto de reorden.
- En el de **facturación**, es una línea con precio, impuesto y clave del SAT.
- En el de **envíos**, es un bulto con peso, dimensiones y fragilidad.

Intentar un único modelo Producto que sirva para los cuatro produce una clase monstruosa que nadie entiende y que cambia cada vez que cambia cualquiera de los cuatro negocios. La propuesta de DDD es aceptar que son **cuatro modelos distintos** que comparten un identificador.

PARA EL EXAMEN

«¿Cómo se decide dónde partir los microservicios?» La respuesta esperada: **por contextos delimitados del dominio, no por capas técnicas**. Un servicio de «pedidos» es correcto; un servicio de «base de datos» o de «controladores» es un error conceptual, porque reintroduce el acoplamiento que se quería evitar y obliga a modificar varios servicios para cualquier cambio funcional.

Dentro de cada contexto, el equipo y el código usan exactamente las mismas palabras que usa el negocio. Si el área comercial dice «cotización», la clase se llama `Cotizacion`, no `OrderDraft`. Suena trivial y evita una cantidad enorme de malentendidos.

MAPA DE CONTEXTOS

Documenta cómo se relacionan los contextos entre sí. Los patrones que conviene poder nombrar:

PATRÓN	RELACIÓN QUE DESCRIBE
Cliente–proveedor	El de aguas arriba provee; el de aguas abajo negocia sus necesidades
Conformista	El de abajo se traga el modelo del de arriba sin poder negociar
Capa anticorrupción	El de abajo traduce el modelo ajeno para no contaminar el propio
Núcleo compartido	Dos contextos comparten un subconjunto del modelo. Genera acoplamiento; úsalo poco
Servicio abierto	Un contexto publica un protocolo estándar para muchos consumidores

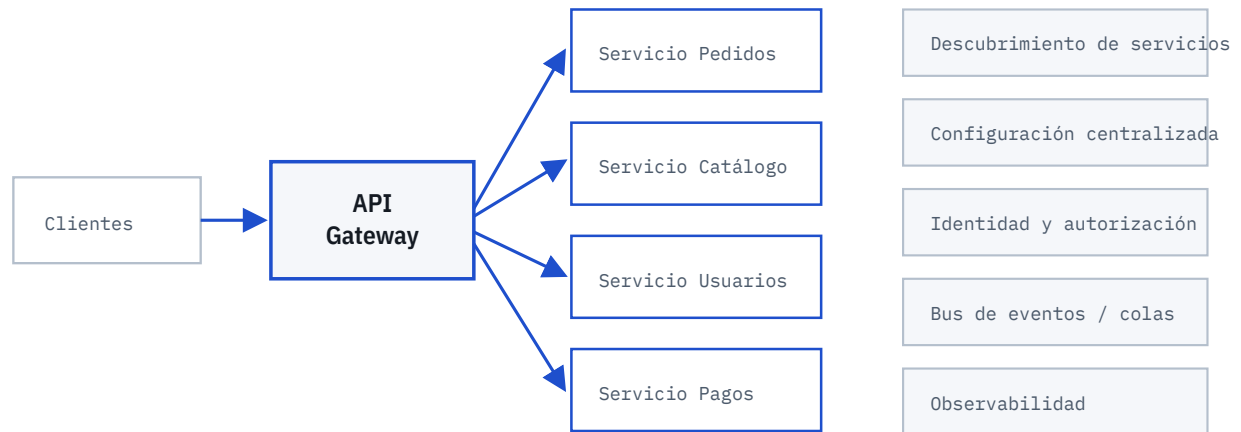
La **capa anticorrupción** merece atención especial porque es la más útil en la práctica: es un adaptador que traduce entre el modelo externo y el tuyo, de modo que si el proveedor cambia su API, el daño se detiene en esa capa. Es exactamente el mismo mecanismo del adaptador hexagonal, aplicado entre servicios.

OTRAS ESTRATEGIAS DE DESCOMPOSICIÓN

- **Por capacidad de negocio** — qué hace la organización: gestión de pedidos, atención al cliente, facturación.
- **Por subdominio** — distinguiendo el subdominio *central* (lo que te hace competitivo, aquí pones tus mejores recursos), los *de soporte* y los *genéricos* (autenticación, facturación: cómpralos hechos).
- **Por volatilidad** — separar lo que cambia semanalmente de lo que cambia una vez al año.

1.6.3 Modelo de referencia

Los componentes de infraestructura que casi cualquier plataforma de microservicios necesita. El programa lo llama «modelo de referencia» porque es el catalogo estándar de piezas.



Transversal a todo: bitácoras correlacionadas, métricas, trazas distribuidas, CI/CD

El gateway es la única puerta pública. Todo lo de la derecha son servicios de plataforma que los servicios de negocio consumen.

COMPONENTE	QUÉ RESUELVE	EJEMPLOS
API Gateway	Punto único de entrada: enrutado, autenticación, límite de tasa, agregación de respuestas, TLS	Kong, NGINX, Spring Cloud Gateway, AWS API Gateway
Descubrimiento	Saber dónde está cada instancia cuando las direcciones cambian solas	Consul, Eureka, DNS de Kubernetes
Balanceador	Repartir carga entre instancias del mismo servicio	NGINX, HAProxy, Service de Kubernetes
Configuración	Configuración externa al artefacto, distinta por entorno	Spring Cloud Config, ConfigMap y Secret
Interruptor	Dejar de llamar a un servicio caído en vez de acumular esperas	Resilience4j, Polly
Bus de eventos	Comunicación asíncrona y desacoplada	RabbitMQ, Kafka, NATS, SQS
Identidad	Autenticación centralizada y emisión de tokens	Keycloak, Auth0, Cognito
Observabilidad	Saber qué pasó cuando algo falla en cinco servicios a la vez	Prometheus, Grafana, Jaeger, OpenTelemetry

EL PATRÓN DE INTERRUPTOR (CIRCUIT BREAKER)

Vale la pena entenderlo porque se pregunta y porque el nombre es literal. Si el servicio de pagos deja de responder, cada petición al servicio de pedidos se queda esperando el tiempo de espera completo, los hilos se agotan y el servicio de pedidos también cae: es un **fallo en cascada**. El interruptor lo evita con tres estados:

- **Cerrado** — funcionamiento normal, las llamadas pasan.
- **Abierto** — tras N fallos consecutivos, las llamadas fallan de inmediato sin intentar la red. Se responde con un valor alternativo o con un error controlado.
- **Semiabierto** — pasado un tiempo, deja pasar una llamada de prueba. Si funciona, vuelve a cerrado; si no, vuelve a abierto.

LOS TRES PILARES DE LA OBSERVABILIDAD

Bitácoras (qué pasó), **métricas** (cuánto y con qué frecuencia) y **trazas** (el recorrido completo de una petición a través de todos los servicios). El elemento que las une es el **identificador de correlación**: un id que se genera en el gateway y viaja en una

cabecera por todas las llamadas subsiguientes, de modo que puedas reconstruir qué hizo una petición concreta en cinco servicios distintos.

1.6.4 Modelo de implementación

COMUNICACIÓN SÍNCRONA FRENTE A ASÍNCRONA

	SÍNCRONA	ASÍNCRONA
Mecanismo	REST sobre HTTP, gRPC, GraphQL	Colas y temas de mensajes, eventos
El que llama	Espera la respuesta, bloqueado	Publica y sigue trabajando
Acoplamiento	Temporal: ambos deben estar arriba	Bajo: el receptor puede estar caído y procesar después
Fallos	Se propagan en cadena	Quedan absorbidos por la cola
Complejidad	Baja, familiar	Alta: orden, duplicados, mensajes muertos
Buena para	Consultas que necesitan respuesta inmediata	Notificar hechos consumados, trabajo diferido

CONSISTENCIA EVENTUAL

Consecuencia directa de tener una base de datos por servicio: no puedes abrir una transacción que abarque dos servicios. El sistema pasa por estados temporalmente inconsistentes y converge después. El pedido queda creado y el inventario se descuenta doscientos milisegundos más tarde; durante ese lapso el sistema está «mal», y hay que diseñar para que eso sea aceptable.

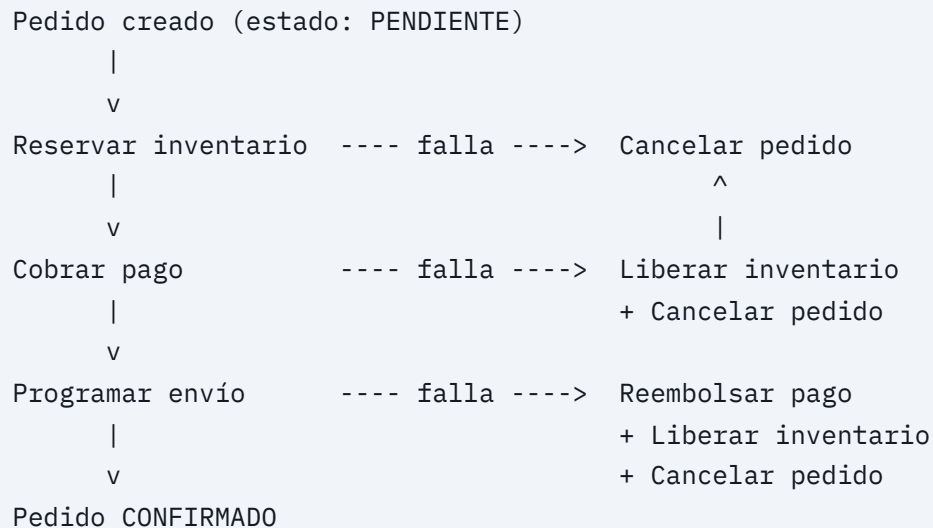
PARA EL EXAMEN

BASE frente a ACID. Frente a las garantías ACID de una base relacional, los sistemas distribuidos suelen ofrecer BASE: *Basically Available* (básicamente disponible), *Soft state* (estado blando, puede cambiar sin entrada nueva) y *Eventually consistent* (consistente con el tiempo). Es el intercambio que impone el teorema CAP en cuanto eliges disponibilidad sobre consistencia.

EL PATRÓN SAGA

Es la respuesta a «¿cómo hago una transacción que toca tres servicios?». Una saga es una secuencia de transacciones locales, cada una en su propio servicio, donde cada paso publica un evento que dispara el siguiente. Si un paso falla, se ejecutan **transacciones de compensación** que deshacen los pasos anteriores.

SAGA DE UN PEDIDO, CON SU CAMINO DE COMPENSACIÓN



Hay dos variantes: la **coreografía**, donde cada servicio escucha eventos y reacciona sin coordinador central (bajo acoplamiento, pero el flujo global no está escrito en ninguna parte y cuesta depurarlo); y la **orquestación**, donde un componente coordinador dirige los pasos (el flujo es explícito y visible, a cambio de un punto central de dependencia).

IDEMPOTENCIA

En una red poco fiable, los mensajes se reintentan y llegan por duplicado. Si «cobrar pago» se ejecuta dos veces, cobraraste dos veces. La solución estándar es la **clave de idempotencia**: el emisor manda un identificador único por operación, el receptor lleva registro de las claves ya procesadas y, si ve una repetida, devuelve el resultado anterior sin volver a ejecutar. Es un tema que reaparece en la Unidad II.

PATRÓN OUTBOX

Problema sutil pero clásico: necesitas guardar el pedido en tu base de datos y publicar un evento en la cola, atómicamente. Si guardas y luego falla la publicación, el resto del sistema nunca se entera. La solución: dentro de la misma transacción de base de datos, escribes el evento en una tabla `outbox`; un proceso aparte lee esa tabla y publica en la cola. Así la atomicidad la garantiza la base de datos local.

Como los servicios se despliegan de forma independiente, en algún momento convivirán la versión vieja de uno con la nueva de otro. De ahí dos disciplinas: **versionar** (`/api/v1/` , o negociación por cabecera) y mantener **compatibilidad hacia atrás** — añadir campos está permitido, quitarlos o renombrarlos no. Las **pruebas de contrato** (Pact es la herramienta típica) verifican automáticamente que el proveedor sigue cumpliendo lo que el consumidor espera, sin necesidad de levantar los dos servicios juntos.

1.6.5 Modelo de despliegue

Aquí el programa empieza a solaparse con la Unidad V, que retomará contenedores y PaaS con más profundidad.

PATRONES DE DESPLIEGUE

- **Un servicio por contenedor** — el estándar actual. Aislamiento, imagen reproducible, arranque rápido.
- **Un servicio por máquina virtual** — más aislamiento, mucho más costo.
- **Sin servidor** (*serverless*) — despliegas funciones y el proveedor gestiona todo lo demás. Excelente para cargas esporádicas; sufre el arranque en frío.
- **Sidecar** — un contenedor auxiliar junto al principal que se encarga de asuntos transversales (TLS, métricas, reintentos). Es la base de las mallas de servicios como Istio o Linkerd.

ORQUESTACIÓN

Con veinte contenedores dejas de poder gestionarlos a mano. Un orquestador — Kubernetes en la práctica, y hay un libro sobre él en tu bibliografía — se encarga de programar contenedores en nodos, reiniciar los que mueren, escalar según carga, descubrir servicios y hacer despliegues progresivos. El vocabulario mínimo: **Pod** (la unidad más pequeña, uno o varios contenedores juntos), **Deployment** (declara cuántas réplicas quieres), **Service** (nombre estable y balanceo hacia esos pods), **Ingress** (entrada HTTP desde fuera del clúster), **ConfigMap** y **Secret** (configuración y credenciales).

ESTRATEGIAS DE PUBLICACIÓN

ESTRATEGIA	CÓMO FUNCIONA
Recreate	Apaga la vieja, levanta la nueva. Hay corte de servicio
Rolling update	Reemplaza instancias poco a poco. Sin corte, pero conviven dos versiones
Blue-green	Dos entornos completos; se conmuta el tráfico de golpe. Reversión instantánea
Canary	Se manda el 5% del tráfico a la versión nueva y se observa antes de continuar

LOS DOCE FACTORES

Metodología para aplicaciones que van a vivir en la nube (Heroku, 2011). No hace falta memorizar las doce, pero sí las que más se preguntan: **configuración en el entorno** y no en el código; **procesos sin estado** para poder escalar horizontalmente; **paridad entre desarrollo y producción**; **bitácoras como flujo de eventos** hacia la salida estándar en lugar de archivos; y **servicios de respaldo como recursos conectables**, de modo que cambiar la base de datos sea cambiar una variable de entorno.

PAAS

El programa exige publicar el proyecto en un PaaS (prácticas 11 y 12). Un PaaS te da ejecución, escalado, red y TLS, y tú solo entregas código o una imagen. Render, Railway, Fly.io, Heroku, Google App Engine, Azure App Service. Todos toman un repositorio de Git o un Dockerfile y te devuelven una URL pública.

EN TU PROYECTO

Despliega un «hola mundo» en un PaaS **en la primera semana**, cuando no hay presión. Establecer la tubería completa — repositorio, contenedor, variables de entorno, base de datos gestionada, dominio — con una aplicación trivial toma una tarde. Hacerlo con el proyecto terminado y a dos días de la entrega es donde se pierden calificaciones cada semestre.

1.6.6 Diseño de aplicaciones web

El cierre de la unidad: cómo se aterriza todo lo anterior en una aplicación web real, que es exactamente lo que vas a entregar.

EL FRONTEND EN UN MUNDO DE MICROSERVICIOS

- **SPA contra el gateway** — lo habitual y lo que deberías hacer. Un cliente React o Angular consume una única URL base, y el gateway reparte hacia dentro.
- **BFF** (*Backend for Frontend*) — un backend a medida por tipo de cliente: uno para web, otro para móvil. Evita que la aplicación móvil tenga que hacer siete peticiones para pintar una pantalla.
- **Micro-frontends** — llevar la división hasta la interfaz, con equipos que despliegan fragmentos de página independientes. Existe, es complejo, y para esta materia basta con saber que existe.

RENDERIZADO

Distinción que conviene manejar de cara a la Unidad IV: en el **renderizado del lado del cliente** el servidor manda un HTML casi vacío y JavaScript construye la interfaz — el arranque inicial es más lento y el SEO sufre, pero la navegación posterior es instantánea. En el **renderizado del lado del servidor** el HTML llega ya construido — mejor para primera carga y para buscadores, a cambio de más trabajo en el servidor. Next.js y Nuxt combinan ambos.

SEGURIDAD TRANSVERSAL

El gateway autentica y valida el token; los servicios internos confían en el gateway o revalidan el token por su cuenta. La tendencia actual es **confianza cero**: cada servicio verifica siempre, porque estar dentro de la red ya no se considera prueba de nada. Todo esto se desarrolla en el punto 2.6 del programa.

LA RECETA PRÁCTICA

Para un proyecto de materia, la ruta que maximiza aprendizaje y minimiza riesgo:

1. Un **monolito modular con arquitectura hexagonal**, dividido internamente por contextos delimitados con carpetas de fronteras estrictas.
2. Una **API REST** bien diseñada como única interfaz de entrada, documentada con OpenAPI.
3. Un **frontend SPA** separado que la consume.
4. Todo **en contenedores**, con Docker Compose para desarrollo local.
5. **Publicado en un PaaS**, como exige el programa.
6. En la exposición, presenta el **diagrama de cómo se extraerían** dos de esos módulos a microservicios: qué base de datos se llevaría cada uno, qué llamadas

pasarían a ser de red, qué se volvería eventualmente consistente.

POR QUÉ ESTO FUNCIONA EN LA EVALUACIÓN

La unidad de competencia dice «*diseña una aplicación web con base en la arquitectura de microservicios*». El punto 6 de esa lista cumple el verbo **diseña** demostrando que entiendes dónde están las fronteras y qué costos implica cruzarlas, mientras entregas un sistema que de verdad funciona el día de la demostración. Un alumno que intenta seis microservicios reales normalmente llega con tres a medias y ninguno desplegado.

ANTIPATRONES QUE HAY QUE PODER NOMBRAR

- **Monolito distribuido** — servicios separados que comparten base de datos o que deben desplegarse juntos. Lo peor de ambos mundos.
- **Nanoservicios** — servicios tan pequeños que el costo de comunicación y operación supera cualquier beneficio.
- **Base de datos compartida** — ya mencionado, y el más frecuente.
- **Cadena de llamadas** — A llama a B que llama a C que llama a D. La latencia se suma y la probabilidad de fallo se multiplica.
- **Microservicios sin automatización** — desplegar veinte servicios a mano es insostenible.
- **Empezar por microservicios** — trazar fronteras antes de conocer el dominio garantiza trazarlas mal.

PARTE 3

De la teoría a tu proyecto

La Unidad I no se evalúa solo por escrito. El programa exige un proyecto publicado y verificable, y las decisiones que tomes en las primeras dos semanas te acompañan todo el semestre.

3.1 Las decisiones que se toman una sola vez

DECISIÓN	CUÁNDO TOMARLA	QUÉ PASA SI LA CAMBIAS DESPUÉS
Lenguaje y framework	Semana 1	Reescritura completa
Monolito o microservicios	Semana 1–2	Semanas de trabajo
Estructura de carpetas	Semana 2	Doloroso pero factible
Relacional o documental	Semana 2–3	Migración de datos y de repositorios
Esquema de autenticación	Unidad II	Moderado si aislaste bien
Proveedor de PaaS	Semana 1	Bajo, si contenerizaste
Librería de interfaz	Unidad IV	Reescritura del frontend

Las tres primeras son arquitectura en el sentido estricto de la Parte 2: caras de revertir. Las últimas son detalles — y que sean detalles es precisamente el resultado de haber aplicado bien los puntos 1.4 y 1.5.

Esqueleto recomendado

ESTRUCTURA COMPLETA DEL PROYECTO

```
proyecto/  
  backend/  
    src/  
      contextos/  
        pedidos/  
          dominio/      Pedido.js Linea.js errores.js  
          aplicacion/   CrearPedido.js ConfirmarPedido.js  
          puertos/      RepositorioPedidos.js PasarelaPagos.js  
          infraestructura/ RepositorioPedidosPg.js rutas.js  
        catalogo/  
          dominio/ aplicacion/ puertos/ infraestructura/  
        usuarios/  
          dominio/ aplicacion/ puertos/ infraestructura/  
        compartido/     errores base, tipos, utilidades  
        config/          variables de entorno, conexiones  
        main.js          el ensamblador  
      tests/  
        unit/            dominio y casos de uso, sin infraestructura  
        integration/     repositorios contra base real  
      Dockerfile  
      .env.example       NUNCA subas el .env real  
      package.json  
    frontend/  
      src/  
        componentes/  
        paginas/  
        servicios/      clientes HTTP hacia la API  
        hooks/  
      Dockerfile  
      package.json  
    docker-compose.yml   api + front + base de datos  
    docs/  
      arquitectura.md    diagramas y decisiones  
      api.yaml           especificación OpenAPI  
    README.md
```

Lo importante de esa estructura: el primer nivel dentro de `src/` son **contextos del negocio**, no capas técnicas. Dentro de cada contexto sí aparecen las capas. Así la cohesión es alta por contexto, y si algún día extraes uno a microservicio, te llevas una carpeta completa.

Registro de decisiones de arquitectura

Un ADR (*Architecture Decision Record*) es un documento corto por cada decisión importante. El programa pide «reportes de indagación» y «reporte de proyecto»; los

ADR encajan perfectamente y cuestan diez minutos cada uno.

```
DOCS/ADR/002-MONOLITO-MODULAR.MD
```

```
# ADR 002 – Monolito modular en lugar de microservicios
```

```
### Estado
```

```
Aceptada – 2026-02-10
```

```
### Contexto
```

```
Equipo de tres personas, un semestre de plazo, sin experiencia previa  
operando sistemas distribuidos. El programa exige publicar la aplicación  
en un PaaS y demostrar diseño con base en microservicios.
```

```
### Decisión
```

```
Construir un monolito modular con arquitectura hexagonal, dividido  
internamente en contextos delimitados (pedidos, catálogo, usuarios),  
empaquetado en un contenedor y desplegado en Render.
```

```
### Consecuencias
```

- + Un solo despliegue y un solo pipeline: menos tiempo en operación.
- + Transacciones ACID disponibles dentro de todo el sistema.
- + Fronteras ya trazadas: la extracción futura es viable.
- No demostramos operación real de servicios distribuidos.
Se mitiga con un diseño documentado de la extracción en la exposición.

Qué llevar a la exposición

El programa incluye exposiciones como evidencia evaluable. Cuatro cosas que casi nadie lleva y que se notan:

1. Un **diagrama de contexto** (nivel 1 de C4): tu sistema, sus usuarios y los sistemas externos con los que habla.
2. Un **diagrama de contenedores** (nivel 2): frontend, API, base de datos, servicios de terceros, con los protocolos en cada flecha.
3. El **mapa de contextos delimitados**, explicando por qué cortaste donde cortaste.
4. Uno o dos **ADR** que muestren que la decisión fue razonada y no accidental.

Veinte preguntas del tipo que cae en la evaluación escrita de esta unidad, con respuesta modelo. Inténtalas antes de abrir cada una.

01 ¿Qué es arquitectura de software y en qué se distingue de un patrón de diseño?

La arquitectura es el conjunto de decisiones estructurales de alto nivel sobre un sistema — sus componentes principales, las relaciones entre ellos y los principios que guían su evolución — y se caracteriza por ser costosa de revertir. Un patrón de diseño es una solución reutilizable a un problema recurrente en un ámbito mucho más local, normalmente entre unas pocas clases. La arquitectura decide que el acceso a datos estará aislado; el patrón Repository dice cómo se implementa ese aislamiento.

02 Diferencia entre capa (*layer*) y nivel (*tier*).

Una capa es una división lógica del código por responsabilidad; un nivel es una división física en procesos o máquinas distintas. Una aplicación de tres capas puede ejecutarse en un solo nivel — todo en una máquina — o repartirse en tres niveles: navegador, servidor de aplicación y servidor de base de datos. En español ambos términos se traducen «capa», de ahí la confusión.

03 ¿Por qué se dice que en n-capas la base de datos es el centro de gravedad?

Porque las dependencias apuntan hacia abajo: presentación depende del negocio y el negocio depende del acceso a datos. En consecuencia, las decisiones de persistencia se propagan hacia arriba, el negocio no se puede probar sin base de datos, las entidades del ORM se filtran hacia las capas superiores y el diseño tiende a empezar por el esquema de tablas en lugar de por el modelo del dominio.

04 Enuncia la regla de dependencia de la arquitectura limpia.

Las dependencias del código fuente solo pueden apuntar hacia adentro, hacia políticas de mayor nivel. Ningún elemento de un anillo interior puede conocer nada de un anillo exterior: ni nombres de clases, ni funciones, ni variables, ni formatos de datos.

05 ¿Cómo puede el dominio usar una base de datos sin depender de ella?

Mediante inversión de dependencias. El dominio declara una *interfaz* de repositorio expresada en su propio vocabulario, y la capa de infraestructura provee una *implementación* concreta que usa el ORM o el SQL. En tiempo de ejecución, el ensamblador inyecta la implementación. Así la flecha de dependencia del código fuente va de infraestructura hacia dominio, aunque el flujo de control vaya en sentido contrario.

06 ¿Por qué la arquitectura hexagonal se dibuja como un hexágono?

Por conveniencia de dibujo, no por significado. Cockburn quería representar que la aplicación tiene múltiples puntos de entrada y salida alrededor de su núcleo, y una figura con lados rectos permite colocarlos con claridad. El número seis no corresponde a seis capas ni a seis tipos de puerto.

07 Diferencia entre puerto primario y puerto secundario.

Un puerto primario o de entrada es la interfaz por la que el exterior invoca a la aplicación; la aplicación lo define y lo implementa mediante sus servicios de aplicación. Un puerto secundario o de salida es la interfaz que la aplicación necesita para hablar con el exterior; la aplicación lo define pero lo implementa un adaptador de infraestructura. La asimetría — definir ambos, implementar solo los primarios — es lo que materializa la inversión de dependencias.

08 ¿Qué tienen en común y en qué difieren cebolla, limpia y hexagonal?

Comparten el principio central: el dominio en el centro, dependencias apuntando hacia adentro y aislamiento de los detalles técnicos mediante inversión de dependencias. Difieren en metáfora y énfasis: la hexagonal subraya la simetría entre quien nos llama y a quien llamamos con puertos y adaptadores, e incluye las pruebas como un adaptador más; la de cebolla subraya que el modelo del dominio, y no la base de datos, ocupa el centro, y sitúa las interfaces de repositorio dentro del dominio; la limpia enuncia la regla de dependencia de forma explícita y nombra cuatro anillos.

09 ¿Qué es un monolito modular y por qué no es lo mismo que una bola de lodo?

Un monolito modular se despliega como una sola unidad pero está organizado internamente en módulos con fronteras explícitas que se comunican por interfaces definidas. Una bola de lodo también se despliega como una unidad, pero carece de

fronteras: cualquier parte accede a cualquier otra. Lo que define al monolito es la unidad de despliegue, no la calidad interna; confundir ambos conceptos es el error más común del tema.

10 Los dos sentidos de la palabra «contenedor».

El contenedor de aplicación es un entorno de ejecución que aloja la aplicación y le provee ciclo de vida, hilos, pool de conexiones, seguridad y transacciones — Tomcat, WildFly — operando por inversión de control. El contenedor de sistema operativo es un empaquetado aislado con la aplicación y sus dependencias que comparte el kernel del anfitrión — Docker — y resuelve la reproducibilidad del entorno. Un monolito empaquetado en Docker sigue siendo un monolito.

11 Diferencias entre contenedor y máquina virtual.

La máquina virtual virtualiza hardware completo y ejecuta su propio sistema operativo, lo que implica gigabytes de tamaño y decenas de segundos de arranque, a cambio de un aislamiento muy fuerte. El contenedor virtualiza a nivel de sistema operativo mediante *namespaces* y *cgroups*, comparte el kernel del anfitrión, pesa megabytes y arranca en segundos, con un aislamiento bueno pero menor.

12 Define microservicios y da tres características.

Estilo arquitectónico en el que la aplicación es un conjunto de servicios pequeños y autónomos, cada uno en su propio proceso, desplegados de forma independiente y comunicados por mecanismos ligeros. Tres características: organización en torno a capacidades de negocio en lugar de capas técnicas; gestión de datos descentralizada, con una base de datos por servicio; y diseño para el fallo, asumiendo que los servicios remotos fallarán.

13 ¿Qué es un monolito distribuido y por qué es un antipatrón?

Un conjunto de servicios físicamente separados pero fuertemente acoplados: comparten base de datos, o deben desplegarse en conjunto, o cualquier cambio funcional obliga a modificar varios a la vez. Es un antipatrón porque paga todos los costos de un sistema distribuido — latencia, fallos parciales, complejidad de despliegue y depuración — sin obtener el beneficio principal, que es la independencia de despliegue y evolución.

14 ¿Con qué criterio se decide dónde dividir los microservicios?

Por contextos delimitados del dominio, siguiendo DDD: fronteras dentro de las cuales un modelo y su vocabulario son consistentes. También por capacidades de negocio o por volatilidad. Nunca por capas técnicas, porque un servicio «de acceso a datos» obligaría a modificar varios servicios ante cualquier cambio funcional, reintroduciendo el acoplamiento que se buscaba eliminar.

15 ¿Por qué cada microservicio debe tener su propia base de datos?

Porque compartir base de datos crea acoplamiento a nivel de esquema: ningún servicio puede evolucionar su modelo de datos sin coordinar con los demás, y se pierde la independencia de despliegue que justifica toda la arquitectura. Además permite que cada servicio elija la tecnología de persistencia adecuada a su problema. El costo es renunciar a las transacciones distribuidas y a los joins entre servicios.

16 ¿Qué es el patrón saga y para qué sirve?

Es la alternativa a las transacciones distribuidas: una secuencia de transacciones locales, cada una en su propio servicio, donde cada paso dispara el siguiente. Si un paso falla, se ejecutan transacciones de compensación que deshacen los efectos de los pasos anteriores. Se implementa por coreografía — cada servicio reacciona a eventos, sin coordinador — o por orquestación, con un coordinador que dirige la secuencia.

17 Explica el teorema CAP y su consecuencia práctica.

En un sistema distribuido no se pueden garantizar simultáneamente consistencia, disponibilidad y tolerancia a particiones de red; a lo sumo dos. Como las particiones de red son inevitables en cualquier red real, la tolerancia a particiones no es opcional, de modo que la elección real es entre consistencia y disponibilidad. Optar por disponibilidad conduce a la consistencia eventual y al modelo BASE.

18 ¿Qué hace un API Gateway?

Es el punto único de entrada a un sistema de microservicios. Enruta cada petición al servicio correspondiente y concentra asuntos transversales: terminación TLS, autenticación y validación de tokens, límite de tasa, agregación de respuestas de

varios servicios, traducción de protocolos y registro de bitácoras. Evita que el cliente conozca la topología interna y que cada servicio reimplemente lo mismo.

19 ¿Por qué la testabilidad mejora con arquitectura hexagonal?

Porque el dominio y los servicios de aplicación dependen solo de interfaces, nunca de implementaciones concretas. Eso permite sustituir cada puerto secundario por un doble de prueba — típicamente un repositorio en memoria — y probar toda la lógica de negocio sin base de datos, sin red y sin servidor web. Las pruebas resultan rápidas, deterministas y aisladas. En el enfoque de Cockburn, la propia suite de pruebas es un adaptador primario más.

20 ¿Cuándo *no* conviene usar microservicios?

Cuando el equipo es pequeño, cuando el dominio aún no se conoce lo bastante para trazar fronteras estables, cuando no existe automatización de despliegue y observabilidad, cuando el sistema no tiene requisitos de escalado que lo justifiquen, o cuando el proyecto está empezando. La recomendación habitual es «monolith first»: construir un monolito modular, descubrir las fronteras reales del dominio y extraer servicios cuando exista una razón concreta — escalado independiente, equipos separados o ciclos de vida distintos.

PARTE 5

Glosario

Los términos de la unidad en una sola línea cada uno, para repasar la víspera.

ACID

Atomicidad, consistencia, aislamiento y durabilidad. Las garantías de una transacción en base de datos relacional.

Adaptador

Código que conecta un puerto con una tecnología concreta. Primario si nos invoca, secundario si lo invocamos.

ADR

Documento breve que registra una decisión de arquitectura, su contexto y sus consecuencias.

Agregado	Grupo de entidades y objetos de valor tratado como una unidad de consistencia, con una raíz como única puerta de entrada.
API Gateway	Punto único de entrada que enruta y concentra asuntos transversales en un sistema de microservicios.
BASE	Básicamente disponible, estado blando, consistencia eventual. El modelo típico de los sistemas distribuidos.
BFF	Backend a la medida de un tipo de cliente concreto, para evitar que el cliente encadene muchas peticiones.
Bola de lodo	Sistema sin fronteras internas donde cualquier parte accede a cualquier otra.
CAP	Teorema según el cual un sistema distribuido garantiza a lo sumo dos de consistencia, disponibilidad y tolerancia a particiones.
Capa anticorrupción	Adaptador que traduce el modelo de un sistema externo al propio para no contaminarlo.
Consistencia eventual	El sistema converge a un estado consistente tras un intervalo, en lugar de garantizarlo al instante.
Contexto delimitado	Frontera dentro de la cual un modelo y su vocabulario son consistentes y sin ambigüedad.
Ensamblador	El único punto del programa donde se instancian las clases concretas y se inyectan las dependencias.
Entidad	Objeto de dominio con identidad propia que persiste aunque cambien sus atributos.
Idempotencia	Propiedad de una operación cuyo efecto es el mismo si se ejecuta una o muchas veces.
Interruptor	Patrón que corta las llamadas a un servicio caído para evitar fallos en cascada.

Inversión de control	El framework o contenedor controla el flujo y llama a tu código, no al revés.
Lenguaje ubicuo	Vocabulario compartido entre negocio y código dentro de un contexto delimitado.
Monolito distribuido	Servicios separados pero acoplados. Todos los costos de lo distribuido, ninguna ventaja.
N+1	Problema de rendimiento en que un ORM lanza una consulta por cada elemento de una colección ya cargada.
Objeto de valor	Objeto inmutable sin identidad, definido enteramente por sus atributos.
Outbox	Patrón que garantiza atomicidad entre guardar en base de datos y publicar un evento.
PaaS	Plataforma como servicio: entregas código o una imagen y el proveedor gestiona ejecución, escalado y red.
Puerto	Interfaz que define la aplicación para comunicarse con el exterior, expresada en vocabulario de dominio.
Regla de dependencia	Las dependencias del código fuente solo apuntan hacia adentro, hacia políticas de mayor nivel.
Repositorio	Abstracción del almacenamiento como si fuera una colección de objetos de dominio en memoria.
Saga	Secuencia de transacciones locales con compensaciones, alternativa a la transacción distribuida.
Sidecar	Contenedor auxiliar que corre junto al principal y asume responsabilidades transversales.

Plan de dos semanas

Si tienes dos semanas antes de que arranque el semestre, este es el orden que rinde más por hora invertida.

Semana 1 — los cimientos

DÍA	TEMA	ENTREGABLE PROPIO
1	HTTP a fondo: métodos, códigos, cabeceras, idempotencia	Media hora leyendo peticiones reales en la pestaña Network
2	JavaScript asíncrono y módulos, o repaso de Java y Maven	Un script que consuma una API pública y maneje errores
3	SOLID con ejemplos propios, con énfasis en inversión de dependencias	Un ejemplo escrito de cada letra, con código tuyo
4	API REST CRUD con Express o Spring Boot	Cinco endpoints con códigos de estado correctos
5	Postman: colecciones, variables de entorno, pruebas	Una colección que ejercite tu API completa
6	SQL y modelo relacional: joins, 1:N, N:M, transacciones	Un esquema de cuatro tablas con sus relaciones
7	Docker: Dockerfile, build, run, compose	Tu API y su base de datos levantando con un solo comando

Semana 2 — la unidad

DÍA	TEMA	ENTREGABLE PROPIO
8	Principios de arquitectura: cohesión, acoplamiento, atributos de calidad	Diagrama de un proyecto tuyo anterior, con sus flechas
9	Monolito y n-capas: ventajas, límites, estructura	Reorganiza tu API del día 4 en tres capas
10	Cebolla y limpia: la regla de dependencia	Extrae el dominio a su propia carpeta, sin imports externos
11	Hexagonal: puertos y adaptadores	Define tus puertos como interfaces y escribe dos adaptadores del mismo puerto
12	Pruebas con dobles	Prueba un caso de uso completo sin levantar la base de datos
13	Microservicios: fundamentos, DDD, comunicación, despliegue	Mapa de contextos de una tienda en línea imaginaria
14	PaaS	Tu API desplegada en Render o Railway, con una URL pública que responde

EL DÍA 14 IMPORTA MÁS DE LO QUE PARECE

Llegar al primer día de clase con una API funcionando, en un contenedor y desplegada en la nube, te pone por delante de las prácticas 1, 2, 3, 4, 11 y 12 del programa — seis de doce. Y libera tu tiempo del semestre para lo que de verdad cuesta: el diseño y el proyecto.

Dónde estudiar cada cosa

FUENTE	PARA QUÉ SIRVE AQUÍ
MDN Web Docs	La referencia de HTTP, CORS y APIs del navegador. Gratuita y exacta
Martin Fowler, artículo «Microservices»	El texto fundacional con Lewis. Corto y citable en un examen
Cockburn, «Hexagonal Architecture»	El original de puertos y adaptadores. Se lee en veinte minutos
Martin, <i>Clean Architecture</i>	La regla de dependencia desarrollada. Los capítulos de la parte V son los relevantes
Newman, <i>Building Microservices</i>	El libro práctico de referencia sobre descomposición y operación
Holmes, <i>Getting MEAN</i>	Está en tu bibliografía oficial. Ruta JavaScript de extremo a extremo
Varanasi y Bartkov, <i>Spring REST</i>	Está en tu bibliografía oficial. Ruta Java para APIs y microservicios
Zammeti, <i>Modern Full-Stack Development</i>	Está en tu bibliografía oficial. Cubre TypeScript, React, Node, Webpack y Docker juntos
c4model.com	Cómo dibujar los diagramas que vas a necesitar en la exposición
12factor.net	Los doce factores completos. Se lee en media hora

Guía de estudio de la Unidad I – **Web client and backend development frameworks** (C704)

Ingeniería en Sistemas Computacionales, plan 2020 · ESCOM y UPIIZ, Instituto Politécnico Nacional

Contenidos y numeración tomados del programa de estudios vigente a partir de enero de 2023.

Las recomendaciones de stack, proyecto y plan de estudio son sugerencias, no parte del programa oficial.